

Error Understanding in Program Code: A Systematic Study of LLM-DL Combinations for Multi-label Classification

Md Faizul Ibne Amin, *Member, IEEE*, Yutaka Watanobe, *Member, IEEE*, Md. Mostafizer Rahman, Daniel M. Muepu, *Graduate Student Member, IEEE*, and Md. Shahajada Mia

Abstract—Programming is a core skill in computer science and software engineering (SE), yet identifying and resolving code errors remains challenging for both novice and experienced developers. Errors rarely occur in isolation, which makes error understanding a multi-label problem. Large Language Models (LLMs) have shown remarkable capabilities in natural language (NL) understanding and generation tasks, but how code-specialized LLMs behave when paired with deep learning (DL) sequence decoders, and which component of such a pipeline drives performance, remains insufficiently explored. This study presents a systematic evaluation of LLM-DL combinations for multi-label error classification (MLEC) of source code. Eight fine-tuned LLMs, including CodeT5, GraphCodeBERT, CodeT5+, UniXcoder, RoBERTa, RoBERTa with a narrowed learning-rate range (RoBERTa LR), PLBART, and CoText, are integrated with GRU, LSTM, BiLSTM, and BiLSTM with an additive attention mechanism (BiLSTM-A) decoders on a real-world student-written Python code error dataset. The resulting 32 model variants, tuned with Optuna, are assessed on a comprehensive multi-label metric suite spanning average accuracy, exact match accuracy, one-error, macro and weighted precision, recall and F1-score, Hamming loss, Jaccard similarity, and ROC-AUC. In single-run evaluation, CodeT5+ GRU performs best, with a weighted F1-score of 0.8243, average accuracy of 91.84%, exact match accuracy of 53.78%, Hamming loss of 0.0816, and one-error of 0.0708. To identify where this performance originates, seed-controlled baselines and component ablations are added with paired significance testing. Encoder choice has the largest effect: across four encoders sharing an identical linear classification head, the weighted F1-score spans 0.7846 to 0.8263, ordered by code specialization. On CodeT5+, the linear head exceeds the GRU hybrid under matched seeds by 0.0040 weighted F1 ($p = 0.0013$) while training about 24% faster. Max pooling outperforms mean and attention pooling, and explicitly modeling label interactions does not improve weighted F1 despite substantial label co-occurrence. These results identify encoder quality, rather than decoder complexity, as the primary lever for MLEC and support the development of scalable automated feedback tools for programming education (PE) and SE.

Index Terms—Multi-label classification, error understanding, LLM, DL, CodeT5, GraphCodeBERT, CodeT5+, UniXcoder, RoBERTa, PLBART, CoText, GRU, LSTM, BiLSTM, BiLSTM-A, ablation study, programming learning, software engineering

I. INTRODUCTION

PROGRAMMING underpins various domains such as AI, data analysis, and modern applications, making it essential not only for Information and Communication Technology (ICT) students but also for learners across disciplines to develop computational and innovative thinking [1]. However, many beginners struggle with problem-solving and implementation due to the dual demands of declarative and procedural knowledge [2]. Traditional programming courses often fall

short because of limited duration, insufficient practice, and a lack of personalized feedback. E-learning and Online Judge (OJ) systems address this by offering extensive practice and supporting individual learning [3], and many universities now employ Automated Program Assessment (APA) systems to enhance instructional effectiveness [4]. Despite these advances, programmers, particularly novice students, frequently encounter errors, especially logic errors that conventional compilers cannot detect because they allow code to run but produce incorrect results. Understanding such errors remains a key challenge for both students and educators, particularly as real-world programming problems become increasingly complex [5]–[8].

Multi-label classification (MLC) has emerged as a prominent research paradigm, in which each instance may be associated with several label categories simultaneously [9], [10], and has been applied across text categorization [11], biomedical research [12], and multimodal tasks spanning text, images, and sentiment [13]–[15]. Unlike single-label settings, MLC must capture inter-label correlations and frequent co-occurrence patterns over an exponential label space [16]. This is especially demanding in Natural Language Processing (NLP), where models encode fine-grained semantics while accounting for label dependencies [17]. Established approaches include problem transformation [18], algorithm adaptation [19], neural architecture design [20], and ensemble strategies [21]. Class imbalance, intricate decision surfaces, and overlapping label semantics nonetheless persist, particularly in large-scale and education-oriented applications [7], [22].

Transformer-based LLMs have achieved state-of-the-art (SOTA) performance across a wide range of NLP and code-related tasks. Trained on vast corpora that include code, these models have demonstrated strong capabilities in code generation [23], summarization [24], translation [25], completion [26], program repair and classification [27]–[29]. Their applications now extend to educational contexts, such as aid in generating coding exercises, explaining code, and helping students understand error messages. Fine-tuning or integrating LLMs for deeper code understanding and precise error analysis nonetheless remains an active research area [30]–[32]. In parallel, DL models, specifically Recurrent Neural Networks (RNNs) such as GRUs, LSTMs, and BiLSTMs, model sequential patterns and token-level dependencies within source code, extracting hierarchical and temporal features relevant to program structure and semantics.

Generative LLMs (e.g., GPT) excel in reasoning-intensive tasks such as generation, translation, and repair. For classification, and particularly for MLEC, encoder-based or code-specialized LLMs are generally more suitable because they produce discriminative embeddings for downstream classification tasks. Building upon these insights, we focus on LLMs trained or adapted for programming-related and NL reasoning tasks, including CodeT5, CodeT5+, GraphCodeBERT, UniXcoder, RoBERTa, RoBERTa_LR, PLBART, and CoTexT, which align closely with our objective. Throughout, LLM refers to these pre-trained transformer models, all of which are encoder-scale rather than generative systems. Although LLMs have proven effective in many code-related tasks, their integration with DL architectures for MLEC has not yet been systematically compared in PE and SE. This study addresses that by evaluating a unified LLM-DL architecture in which pre-trained LLM encoders are paired with recurrent decoders, namely GRU, LSTM, BiLSTM, and BiLSTM-A, across a controlled set of configurations.

In this architecture, the LLM functions as the primary encoder, transforming input code sequences into contextual representations. These representations are processed by recurrent decoders capable of modeling sequential and structural relationships in code, followed by classification layers that produce multi-label predictions. Beyond measuring the performance of these combinations, this study asks which component of the pipeline determines it, and reports controlled comparisons that isolate the contribution of the encoder, the recurrent decoder, and the pooling and label-modeling choices. The contributions of this study are as follows.

- We present a systematic study of 32 configurations pairing eight pre-trained LLM encoders with four recurrent DL decoders for MLEC of program code, tuned with Optuna and evaluated on a comprehensive multi-label metric suite over a real-world Python code error dataset.
- We establish seed-controlled baselines in which each encoder is paired with a simple linear classification head. Reporting mean and standard deviation across seeds, with paired significance testing, isolates the contribution of the recurrent decoder from that of the encoder.
- We ablate the components of the architecture under a single protocol: removing the recurrent decoder, comparing three pooling strategies, and adding a head that models interactions among labels explicitly.
- We characterize the dataset and the computational cost of the leading configurations, reporting label cardinality, density, per-label support, and pairwise label co-occurrence, together with training time and inference throughput.

Taken together, these results show that encoder choice is the primary determinant of MLEC performance. With a strong code-specialized encoder, a simple classification head matches or exceeds the recurrent hybrid at lower computational cost, while the recurrent decoder appears more relevant when encoder representations are weaker.

The rest of the article is organized as follows: Section II reviews related studies, and Section III presents the task de-

scription and motivation. Section IV describes the integration methodology, while Sections V and VI present the experimental setup and results. Section VII discusses the findings, and Section VIII concludes the study.

II. RELATED WORK

Supporting programmers in understanding, identifying, interpreting, and categorizing source code errors has become a promising area of research in both SE and PE [7]. Research [33] analyzes large-scale coding data from an OJ system using logs, code files, and test cases to extract latent programming knowledge. Another study [34] identifies algorithms in program code using a CNN model to extract structural features. A multi-layered Bi-LSTM model with optimized hyperparameters [35] classifies multilingual source code, addressing heterogeneous codebases in PE and SE. A rule-based error classification tool [6] analyzes frequent error patterns between novice and expert programmers to categorize errors in incorrect and corrected code pairs. Additional studies address the programming learning experience more broadly [36]–[38].

MLC has achieved strong performance across NLP domains and continues to expand into new application areas [39], [40]. In research [31], a Neural Expectation-Maximization (nEM) framework handles noisy labels in multi-label text classification by combining neural encoders with EM optimization, improving results in both single- and multi-instance settings. An Attention-Augmented Memory Network [15] integrates categorical memory, channel-relation, and spatial-relation modules for image MLC. Research [41] developed BoostXML, a DL-based extreme MLC method that improves tail-label prediction using gradient boosting, corrective alignment, and pretraining strategies. Other approaches model relationships among labels directly, exploiting high-order label correlations to improve prediction [42]. In the context of error classification, Amin et al. [5] performed MLC by combining CodeT5 with ML-KNN, demonstrating competitive results. Further, Amin et al. [7] employed fine-tuned BERT variants (uncased and cased) for the MLEC task and compared the results with the baselines, including LLMs (CodeT5, CodeBERT) with ML classifiers (Decision Tree, Random Forest, Ensemble Learning, and ML-KNN). Several more studies in related areas can be found [43]–[45].

In recent years, transformer-based LLMs have become the SOTA for diverse NLP tasks [46]. Notable models include CodeT5 [47], BERT [48], GraphCodeBERT [49], CodeT5+ [50], GPT [51], UniXcoder [52], RoBERTa [53], PLBART [54], CodeLLama [55], and CoTexT [56], which have shown significant superiority in programming and SE-related tasks. In parallel, recurrent models such as GRU, LSTM, and BiLSTM [57] and the attention mechanisms that followed them [58], [59] underpin much of the sequential modeling used in syntax-dependent tasks. Research [60] proposes an LLM-based approach for algebra error classification in student responses, outperforming rule-based and syntax-tree-dependent methods. MWPE-

300K [61] is a large-scale dataset of 304K math reasoning errors from 15 LLMs, paired with a dynamic error classification framework that improves performance through error-aware prompting. A further study [62] presents a DL approach for binary and multiclass myocardial infarction detection, showing that data balancing significantly improves performance, with a DNN reaching competitive accuracy. More recent work on LLM and DL is presented in [63]–[65].

Although program code understanding, error classification, and multi-label learning have each advanced considerably, their intersection remains only partly mapped. Transformer-based encoders learn contextual and semantic representations of code, while recurrent DL models capture sequential dependencies among tokens, yet the two are typically studied in isolation and their combination has not been systematically compared for MLEC. Prior work in this direction has paired code embeddings with classical classifiers [5] and fine-tuned a general-purpose encoder for the task [7], leaving open how code-specialized encoders behave alongside recurrent decoders, and which component of such a pipeline determines performance. This study addresses that question by evaluating a unified LLM-DL architecture across multiple code-oriented encoders and recurrent decoders, together with controlled comparisons that isolate the contribution of each component.

III. MOTIVATION AND TASK DESCRIPTION

Conventional compilers and static analysis tools are effective in identifying syntactic and type-level errors, such as undefined variables or unmatched parentheses. However, they are far less effective at uncovering *semantic-level logic errors*, which allow the code to compile and run but result in incorrect or unintended behavior [66]. For instance, incorrect loop bounds, misuse of comparison operators, or misapplied functions can lead to incorrect outputs. Let a logic error be denoted as an incorrect transformation in the semantic behavior of a program c_i , such that $compile(c_i) = success$, $run(c_i) = success$, and $output(c_i) \neq expected(c_i)$. Identifying such anomalies requires interpreting program intent, control flow, and contextual semantics, which lie beyond the reach of static, rule-based checking.

While automated assessment platforms, advanced debugging tools, and modern LLMs have expanded the scope of feedback available to programmers, a single erroneous submission often carries several interacting error types at once, and assigning all of them demands multi-label reasoning rather than a single verdict. Earlier work approached this requirement from two directions. In [5], code embeddings were classified by a classical multi-label learner. In [7], a general-purpose encoder was fine-tuned end-to-end for the task. Both established that pre-trained representations carry usable error signal. Neither examined what happens when a code-specialized encoder is followed by a decoder that models token order and structure explicitly, which is the arrangement studied here. Building such a pipeline raises a further question that the design alone cannot answer: how much of the resulting accuracy comes from the encoder, and how much from the decoder placed on top of it. The answer matters in practice as

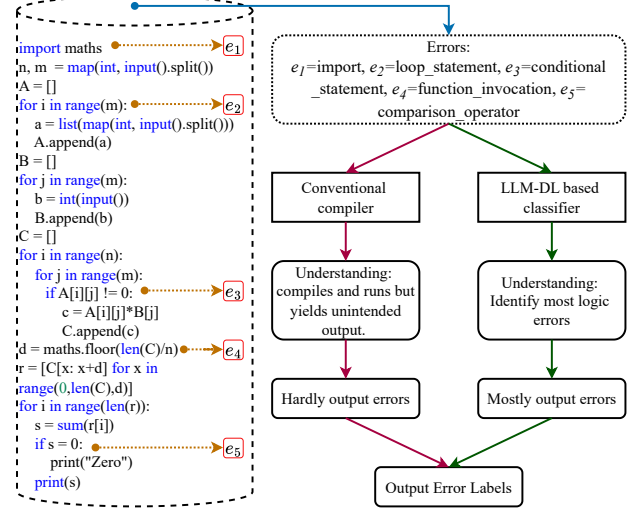


Fig. 1. Source code error understanding and motivational example for the MLEC task

well as in principle, since deploying feedback at scale depends on knowing which part of the model the accuracy is paying for.

This study evaluates a deep neural architecture that pairs pre-trained LLMs, namely CodeT5 [47], GraphCodeBERT [49], CodeT5+ [50], UniXcoder [52], RoBERTa and RoBERTa_LR [53], PLBART [54], and CoTexT [56], with sequence learners GRU, LSTM, BiLSTM, and BiLSTM-A. The LLM encoders produce contextual representations of source code, and the recurrent layers model token-level and structural dependencies across the sequence. Section VI reports what each part of this arrangement contributes.

Let $\mathcal{D} = \{(c_i, \mathbf{y}_i)\}_{i=1}^N$ be a dataset consisting of N code samples, where c_i is a source code snippet and $\mathbf{y}_i \in \{0, 1\}^L$ is a binary indicator vector over L possible error types. The goal of this study is to learn a function $f: \mathcal{C} \rightarrow \mathbb{R}^L$ that maps a code snippet c_i to a logit vector $\mathbf{z}_i = f(c_i)$. The predicted probability vector is then obtained as $\hat{\mathbf{y}}_i = \sigma(\mathbf{z}_i) \in [0, 1]^L$, where each element $\hat{y}_{ij}$ denotes the probability of the j -th error type occurring in c_i . Formally, this can be framed as minimizing the following loss over the training set:

$$\mathcal{L} = \frac{1}{N \cdot L} \sum_{i=1}^N \sum_{j=1}^L \text{BCEWithLogits}(z_{ij}, y_{ij}) \quad (1)$$

where BCEWithLogits denotes the binary cross-entropy loss applied to the raw logit z_{ij} and the ground-truth label y_{ij} for label j of sample i . The example in Figure 1 illustrates the difficulty. The submission compiles and runs, yet carries five distinct error types that compiler diagnostics do not surface. Recovering all of them requires a model that reads semantic and structural cues rather than syntax alone.

IV. COMBINING LLMs WITH DL MODELS

This section describes how LLMs and DL architectures are combined for the MLEC task. The architecture is illustrated

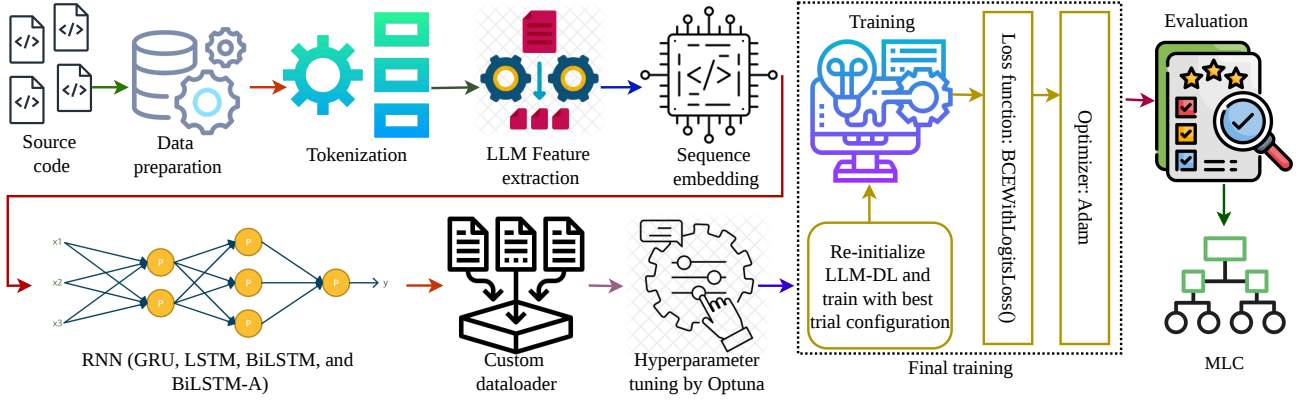


Fig. 2. LLM-DL pipeline for MLEC

in Figure 2, and the training and evaluation pipeline is described in Algorithm 1.

Algorithm 1 MLEC using LLM-DL combinations

- 1: **Input:** Dataset $\mathcal{D} = \{(c_i, \mathbf{y}_i)\}_{i=1}^N$
 - 2: **Output:** Trained model $f : c \mapsto \hat{\mathbf{y}}$ and evaluation metrics
 - 3: **Step 1: Data Preparation**
 - Clean code snippets in $\mathcal{D}$, split into $\mathcal{D}_{\text{train}}$, $\mathcal{D}_{\text{val}}$, and $\mathcal{D}_{\text{test}}$
 - 4: **Step 2: Tokenization**
 - For each $c_i \in \mathcal{D}$ do tokenize c_i using LLM tokenizer:
 - $\text{input_ids}_i \in \mathbb{N}^T$, $\text{attention_mask}_i \in \{0, 1\}^T$
 - end for
 - 5: **Step 3: LLM Feature Extraction**
 - For each batch $(\mathbf{x}, \mathbf{m})$ from DataLoader do:
 - Compute hidden representations using LLM:
 - $\mathbf{H} = \text{LLM}(\mathbf{x}, \mathbf{m}) \in \mathbb{R}^{B \times T \times d}$
 - end for
 - 6: **Step 4: Deep Learning Classifier**
 - Pass $\mathbf{H}$ into RNN module
 - Apply: $\text{RNN} \rightarrow \text{Max Pooling/Attention} \rightarrow \text{Dropout} \rightarrow \text{Dense}$
 - Compute logits:
 - $\mathbf{z} = \mathbf{W}\mathbf{r} + \mathbf{b} \in \mathbb{R}^{B \times L}$
 - 7: **Step 5: Hyperparameter Tuning (Optuna)**
 - Repeat for $t = 1$ to N_{trial} (e.g., $N_{\text{trial}} = 10$) do:
 - Sample trial parameters:
 - $\theta_t = \{\eta, d_h, n_L, p_{\text{drop}}, b_s, d_{\text{bi}}\}$
 - Train f_{θ_t} on $\mathcal{D}_{\text{train}}$ for E_{tune} epochs
 - Evaluate on $\mathcal{D}_{\text{val}}$ and report $F1_{\psi}^{(t)}$ to Optuna
 - end for
 - Select best: $\theta^* = \arg \max_t F1_{\psi}^{(t)}$
 - 8: **Step 6: Final Model Training**
 - Initialize model f_{θ^*} with best hyperparameters
 - Train f_{θ^*} on $\mathcal{D}_{\text{train}}$ for E epochs using Adam
 - Loss function: $\mathcal{L} = \text{BCEWithLogitsLoss}(\mathbf{z}, \mathbf{y})$
 - 9: **Step 7: Evaluation**
 - Evaluate the trained model on $\mathcal{D}_{\text{test}}$
 - Threshold predictions: $\hat{\mathbf{y}} = \sigma(\mathbf{z})$, $\hat{\mathbf{y}} = \mathbb{I}[\hat{\mathbf{y}} > 0.5]$
-

Step 1: Data Preparation Each element of the label vector $\mathbf{y}_i$ indicates the presence or absence of a corresponding error label under the multi-hot encoding used to construct the label matrix. Preprocessing removes unused imports, comments, redundant whitespace, and irrelevant characters while preserving case sensitivity and syntactic structure. The dataset is divided into a development subset $\mathcal{D}_{\text{dev}}$ and a test subset $\mathcal{D}_{\text{test}}$ using an 80:20 split, and 10% of $\mathcal{D}_{\text{dev}}$ is held out for validation. The effective split is therefore 72% training, 8% validation, and 20% testing. Validation data drive Optuna-based tuning and model selection; test data are used only for final evaluation.

Step 2: Tokenization Each code sample c_i is tokenized using the pre-trained tokenizer of the selected LLM, which maps c_i to a fixed-length vector of token IDs and an attention mask:

$$\begin{aligned} \text{input_ids}_i &\in \mathbb{N}^T, \\ \text{attention_mask}_i &\in \{0, 1\}^T \end{aligned} \quad (2)$$

where T is the maximum sequence length. Padding and truncation give uniform input dimensions. Tensors are produced with `return_tensors="pt"`, and the labels $\mathbf{y}_i$ are cast to `FloatTensor`.

Step 3: LLM Feature Extraction Tokenized inputs are passed to the LLM encoder f_{LLM} . Given a batch $(\mathbf{X}, \mathbf{M})$, where $\mathbf{X} \in \mathbb{N}^{B \times T}$ holds the input IDs and $\mathbf{M} \in \{0, 1\}^{B \times T}$ the attention masks, the encoder computes contextual representations:

$$\mathbf{H} = f_{\text{LLM}}(\mathbf{X}, \mathbf{M}) \in \mathbb{R}^{B \times T \times d} \quad (3)$$

where B is the batch size and d is the hidden dimensionality of the LLM. These representations encode token-level semantics and inter-token dependencies. The full sequence of embeddings is retained and passed to the downstream module rather than reduced to a single pooled vector at this stage.

Step 4: DL Classifier The contextual embeddings $\mathbf{H} \in \mathbb{R}^{B \times T \times d}$ are fed into a recurrent sequence model f_{DL} that models dependencies among token embeddings. Let $\mathbf{H}_t \in \mathbb{R}^d$ denote the embedding of the t -th token. The recurrent model produces hidden states:

$$\mathbf{h}_t = f_{\text{RNN}}(\mathbf{h}_{t-1}, \mathbf{H}_t; \theta_{\text{RNN}}), \quad \text{for } t = 1, \dots, T \quad (4)$$

where θ_{RNN} are the trainable recurrent parameters. Under a bidirectional setting, forward and backward hidden states are concatenated:

$$\mathbf{h}_t = [\vec{\mathbf{h}}_t \parallel \overleftarrow{\mathbf{h}}_t] \in \mathbb{R}^{2d_h} \quad (5)$$

where d_h is the hidden size of each unidirectional layer. The hidden-state sequence is aggregated into a sequence-level representation $\mathbf{r}$. For GRU, LSTM, and BiLSTM variants, max pooling is applied over the sequence dimension:

$$\mathbf{r} = \max_{1 \leq t \leq T} \mathbf{h}_t \quad (6)$$

For BiLSTM-A, $\mathbf{r}$ is the attention-weighted context vector produced by the additive attention module. The representation $\mathbf{r}$ passes through a dropout layer and a fully connected layer to produce logits:

$$\mathbf{z} = \mathbf{W}\mathbf{r} + \mathbf{b} \in \mathbb{R}^{B \times L} \quad (7)$$

where $\mathbf{W} \in \mathbb{R}^{L \times d_r}$ and $\mathbf{b} \in \mathbb{R}^L$ are learnable parameters, d_r is the dimensionality of $\mathbf{r}$, and L is the number of labels. Training uses BCEWithLogitsLoss, which operates directly on logits.

Step 5: Hyperparameter Tuning (Optuna) Optuna-based tuning identifies the best configuration for each LLM-DL combination. In each trial t , a configuration is sampled from a predefined search space:

$$\theta_t = \{\eta, d_h, n_L, p_{\text{drop}}, b_s, d_{\text{bi}}\} \quad (8)$$

where η is the learning rate, d_h is the hidden dimension, $n_L \in \{1, 2\}$ is the number of recurrent layers, p_{drop} is the dropout probability, b_s is the batch size, and d_{bi} denotes the bidirectional setting.¹ Each sampled configuration θ_t is trained on $\mathcal{D}_{\text{train}}$ for $E_{\text{tune}} = 1$ epoch and evaluated on $\mathcal{D}_{\text{val}}$. Validation performance is measured by the weighted F1-score, $F1_{\psi}^{(t)}$, and reported to Optuna. The best configuration is then selected as:

$$\theta^* = \arg \max_{t \in \{1, \dots, N_{\text{trial}}\}} F1_{\psi}^{(t)} \quad (9)$$

where $N_{\text{trial}} = 10$ in this study.

Step 6: Final Model Training The model f_{θ^*} is re-initialized with the selected configuration θ^* and trained on $\mathcal{D}_{\text{train}}$ for $E = 20$ epochs using the Adam optimizer, at the tuned learning rate η and the batch size listed in Table III. The model outputs logits $\mathbf{z}$, and the training objective is optimized using BCEWithLogitsLoss. For clarity, this loss can be expressed using the sigmoid-transformed probability $\hat{y}_{ij} = \sigma(z_{ij})$ as:

$$\mathcal{L} = -\frac{1}{B \cdot L} \sum_{i=1}^B \sum_{j=1}^L [y_{ij} \log(\hat{y}_{ij}) + (1 - y_{ij}) \log(1 - \hat{y}_{ij})] \quad (10)$$

where B is the batch size, L is the number of labels, y_{ij} is the ground-truth binary label, and $\hat{y}_{ij}$ is the predicted probability after applying sigmoid to the logit z_{ij} . In implementation, BCEWithLogitsLoss receives logits directly and applies the sigmoid internally in a numerically stable form. Training and validation performance are monitored throughout.

¹The bidirectional flag applies to GRU and LSTM architectures; BiLSTM and BiLSTM-A are inherently bidirectional.

Step 7: Evaluation The trained model is evaluated on $\mathcal{D}_{\text{test}}$. Logits $\mathbf{z}$ are converted to probabilities:

$$\hat{\mathbf{y}} = \sigma(\mathbf{z}) \quad (11)$$

and thresholded at $\tau = 0.5$ to obtain binary multi-label predictions:

$$\tilde{\mathbf{y}} = \mathbb{I}[\hat{\mathbf{y}} > \tau] \quad (12)$$

Test-set performance is then reported using the metrics defined in Section V.

A. Baseline and Ablation Architectures

Section VI compares the LLM-DL combinations described above against the baseline and ablation configurations defined below, all of which share the same encoder, tokenization, loss function, optimizer, data split, and training schedule, and differ only in what follows the encoder. Each configuration receives its own Optuna search under the same budget used for the LLM-DL combinations, so no configuration is evaluated with another’s hyperparameters.

Linear classification head The recurrent decoder is removed. The encoder output $\mathbf{H}$ is aggregated directly over the token dimension by an element-wise maximum, and the pooled vector passes through dropout and a linear projection of the same form as (7):

$$\mathbf{r} = \max_{1 \leq t \leq T} \mathbf{H}_t, \quad \mathbf{z} = \mathbf{W}_h \text{dropout}(\mathbf{r}) + \mathbf{b}_h \quad (13)$$

where $\mathbf{W}_h \in \mathbb{R}^{L \times d}$ and $\mathbf{b}_h \in \mathbb{R}^L$ are learnable parameters. Because the encoder and every other stage of the pipeline are unchanged, the comparison between this configuration and its LLM-DL counterpart measures the contribution of the recurrent decoder.

Pooling variants To examine how the aggregation step affects the head, the maximum in (13) is replaced by two alternatives. Mean pooling averages over non-padding tokens, and additive attention pooling learns a scalar score for each token and returns the weighted sum:

$$\begin{aligned} \mathbf{r}_{\text{mean}} &= \frac{\sum_{t=1}^T m_t \mathbf{H}_t}{\sum_{t=1}^T m_t}, & s_t &= \mathbf{w}_2^\top \tanh(\mathbf{W}_1 \mathbf{H}_t), \\ \alpha_t &= \frac{\exp(s_t)}{\sum_{t'=1}^T \exp(s_{t'})}, & \mathbf{r}_{\text{attn}} &= \sum_{t=1}^T \alpha_t \mathbf{H}_t \end{aligned} \quad (14)$$

where $m_t \in \{0, 1\}$ denotes the attention mask, and $\mathbf{W}_1 \in \mathbb{R}^{d_a \times d}$ and $\mathbf{w}_2 \in \mathbb{R}^{d_a}$ are learnable with $d_a = 128$. Padding positions are excluded from the softmax.

Label-attention head Equations (13) and (14) predict every label from a single shared representation, so dependence among labels is captured only implicitly through the encoder. The label-attention head retains the max-pooled representation of (13) and replaces the final linear layer with a block in which the labels interact directly. The pooled vector is projected into L label-specific embeddings that attend to one another:

$$\begin{aligned} \mathbf{E} &= \text{reshape}_{L \times d_l}(\mathbf{W}_p \text{dropout}(\mathbf{r}) + \mathbf{b}_p), \\ \tilde{\mathbf{E}} &= \text{LayerNorm}(\mathbf{E} + \text{dropout}(\text{MHA}(\mathbf{E}, \mathbf{E}, \mathbf{E}))), \\ z_l &= \mathbf{w}_c^\top \tilde{\mathbf{E}}_l + b_c, \quad l = 1, \dots, L \end{aligned} \quad (15)$$

where $\mathbf{r}$ is the max-pooled encoder output from (13), $\mathbf{W}_p \in \mathbb{R}^{(L \cdot d_l) \times d}$ projects it into L embeddings of size

d_l , and $\tilde{\mathbf{E}}_l \in \mathbb{R}^{d_l}$ is the representation of label l after interaction. MHA denotes multi-head self-attention applied over the label dimension with n_h heads, whose $L \times L$ attention weights represent the learned dependencies among error types, and $\mathbf{w}_c \in \mathbb{R}^{d_l}$ with $b_c \in \mathbb{R}$ form a classifier shared across labels. The pair (d_l, n_h) is tuned jointly over $\{(64, 4), (64, 8), (96, 4), (128, 4), (128, 8)\}$ together with the learning rate, dropout rate, and batch size.

V. EXPERIMENTS

A. Implementation Settings

The experiments were conducted on a 64-bit Windows 11 Pro operating system. The hardware configuration comprised an AMD Ryzen Threadripper 2970WX 24-core processor (3.00 GHz), 64 GB RAM, an NVIDIA GeForce RTX 2080 Ti GPU with 12 GB VRAM, and 1 TB of storage. The 32 LLM-DL combinations in Section VI are each trained once, as in the original evaluation protocol. The baseline and ablation experiments that follow are trained across multiple random seeds, five for the CodeT5+ configurations and three for the supporting encoders, using the seed values 13, 42, 123, 2024, and 31337. For these experiments, results are reported as mean and standard deviation, and paired comparisons across matched seeds are assessed with a two-tailed paired t -test. Optuna tuning for the seeded experiments uses a fixed sampler seed, so each search is reproducible.

B. Dataset and Preprocessing

The error-labeled dataset introduced in [7] is utilized in this study. It consists of 95,631 pairs of erroneous and accepted code samples, collected from 44 introductory programming problems in the “Introduction to Programming 1 (ITP1)” course on the Aizu Online Judge (AOJ) system [67], [68]. Each pair couples a student-submitted erroneous solution with a corresponding accepted solution. Under the original 55-label scheme, each pair contains 3.47 errors on average (± 2.69), with at least one error per pair. As the erroneous code was sourced from AOJ, all submissions were automatically evaluated against test input/output cases, and the corresponding error verdicts were recorded. Basic statistics of the source dataset are shown in Figure 3²

Two preparation steps change these counts before modeling. First, the 55 original labels are summarized into 11 labels by frequency and semantic similarity, which merges errors that share a summarized category. Second, the preprocessing steps in Table I remove duplicate and invalid rows. The resulting dataset contains 82,774 samples carrying 217,363 label occurrences, an average of 2.63 labels per sample (± 1.28), with a minimum of one and a maximum of nine. Label density, the mean fraction of active labels per sample, is 0.239. All experiments in this study use these 82,774 samples, and the 80:20 split yields 16,555 test samples. Further detail on the original and summarized error label sets is available in [5].

Figure 4 shows the frequency of the 11 summarized labels

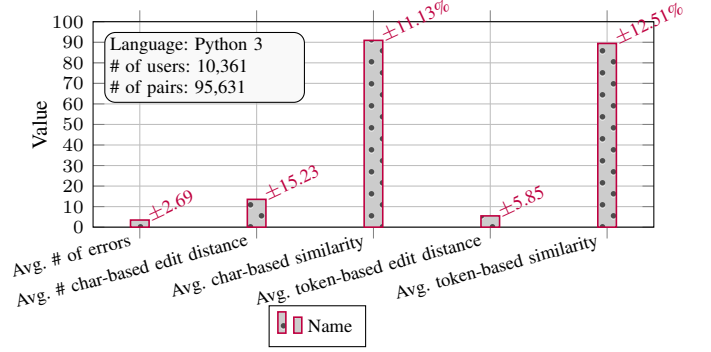


Fig. 3. Basic statistics of the source error label dataset

TABLE I: Data preparation and processing steps

Step	Description	Output
Data Loading	Load dataset containing code samples and error labels	Raw dataset for preprocessing
Data Cleaning	Normalize text by removing extra white lines, comments, and unused imports, preserving syntax/case sensitivity and indentation	Cleaned dataset, optimized for model encoding
Handling Missing Data	Remove empty or invalid rows and verify label-vector consistency	Dataset with no missing/invalid entries
Tokenization	Tokenize code using corresponding LLM tokenizer. Output generated by using <code>return_tensors=pt</code>	input_ids and attention_mask tensors
Label Encoding	Convert labels into a binary indicator matrix using multi-hot encoding scheme	Binary label vectors (1 if present and 0 otherwise)
Dataset Splitting	Split into development (80%) and test (20%); use 10% of the development set for validation	Final subsets: train/validation/test (72%/8%/20%)
Creating DataLoader	Wrap tokenized data and labels into a custom PyTorch dataset	Batches of token-label tensors for train/validation/test

across the preprocessed samples. Support is highly uneven: the most frequent label, `input_output`, occurs in 45,888 samples, whereas the least frequent, `function_definition`, occurs in 454, a ratio of roughly 101 to 1. This imbalance is the principal reason macro-averaged and weighted-averaged results differ throughout Section VI.

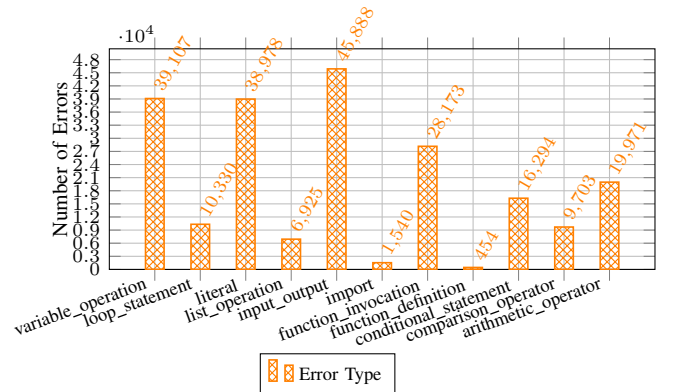


Fig. 4. Frequency of the 11 summarized error labels

Beyond individual frequencies, the labels co-occur

²The Avg. char-based similarity and Avg. token-based similarity values are expressed as percentages, and bar label values are presented in $\pm SD$.

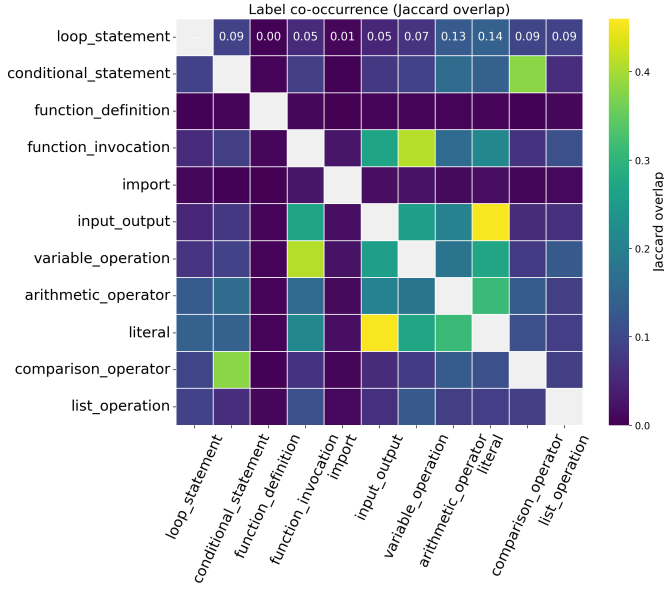


Fig. 5. Pairwise label co-occurrence measured by Jaccard overlap

in semantically coherent ways, and this dependence is concentrated rather than uniform. Measured by the pairwise Jaccard coefficient, three pairs stand clearly above the rest: `input_output` with `literal` (0.455), `function_invocation` with `variable_operation` (0.410), and `conditional_statement` with `comparison_operator` (0.380). The two rarest labels are close to independent of every other label, reaching at most 0.026 for `import` and 0.009 for `function_definition`. As Figure 5 shows, high overlap is restricted to a handful of frequent-label pairs, while the rows for `import` and `function_definition` remain low against every other label. The diagonal is omitted, since self-overlap is always one.

C. Evaluation Metrics

To evaluate model performance across different aspects of prediction quality, the following metric suite is used [7, [42], [69]. Let $\mathbf{y}_i = [y_{i1}, y_{i2}, \dots, y_{iL}] \in \{0, 1\}^L$ represent the ground-truth label vector for the i -th instance, where L is the total number of labels. The model outputs logits $\mathbf{z}_i$, which are converted into probability scores $\hat{\mathbf{p}}_i = \sigma(\mathbf{z}_i) \in [0, 1]^L$. Binary predictions are then obtained as $\tilde{\mathbf{y}}_i = \mathbb{I}[\hat{\mathbf{p}}_i > 0.5]$. All evaluation metrics are computed on the test set containing N instances.

1) *Average Accuracy (AvgAcc)*: *AvgAcc* reflects the mean proportion of correct label-wise predictions per sample, including both TP and TN.

$$\text{AvgAcc} = \frac{1}{N} \sum_{i=1}^N \left(\frac{1}{L} \sum_{j=1}^L \mathbb{I}(y_{ij} = \tilde{y}_{ij}) \right) \quad (16)$$

where $\mathbb{I}(y_{ij} = \tilde{y}_{ij})$ returns 1 when the predicted and ground-truth labels match. Higher values indicate better label-wise correctness.

2) *Exact Match Accuracy (EMAcc)*: *EMAcc* calculates the proportion of instances whose predicted label set exactly matches the true label set.

$$\text{EMAcc} = \frac{1}{N} \sum_{i=1}^N \mathbb{I}(\tilde{\mathbf{y}}_i = \mathbf{y}_i) \quad (17)$$

where $\mathbb{I}(\cdot)$ returns 1 if the full predicted label vector is identical to the ground truth, and 0 otherwise. Higher values indicate stricter multi-label prediction correctness.

3) *One Error (OE)*: *OE* evaluates whether the top-ranked predicted label is not among the true labels. It measures the reliability of the model's most confident prediction.

$$\text{OE} = \frac{1}{N} \sum_{i=1}^N \mathbb{I} \left(\arg \max_j \hat{p}_{ij} \notin \{j \mid y_{ij} = 1\} \right) \quad (18)$$

where $\hat{p}_{ij}$ denotes the predicted probability for label j of instance i and $\arg \max_j \hat{p}_{ij}$ selects the top-scored label. This metric ranges from 0 to 1, with lower values indicating better performance.

4) *Precision (P), Recall (R), and F1 Score (F1)*: These metrics assess the quality of predictions at the label level. Two averaging strategies are used: **macro-average** ($\bar{\mu}$), which treats all labels equally, and **weighted-average** (ψ), which weights each label according to its support.

Macro Averaged Metrics:

$$P_{\bar{\mu}} = \frac{1}{L} \sum_{j=1}^L \frac{TP_j}{TP_j + FP_j} \quad (19)$$

$$R_{\bar{\mu}} = \frac{1}{L} \sum_{j=1}^L \frac{TP_j}{TP_j + FN_j} \quad (20)$$

$$F1_{\bar{\mu}} = \frac{1}{L} \sum_{j=1}^L \frac{2 \cdot TP_j}{2 \cdot TP_j + FP_j + FN_j} \quad (21)$$

Weighted Averaged Metrics:

$$P_{\psi} = \sum_{j=1}^L w_j \cdot \frac{TP_j}{TP_j + FP_j} \quad (22)$$

$$R_{\psi} = \sum_{j=1}^L w_j \cdot \frac{TP_j}{TP_j + FN_j} \quad (23)$$

$$F1_{\psi} = \sum_{j=1}^L w_j \cdot \frac{2 \cdot TP_j}{2 \cdot TP_j + FP_j + FN_j} \quad (24)$$

where TP_j , FP_j , and FN_j denote the true positives, false positives, and false negatives for label j , respectively, and $w_j = \frac{\sum_{i=1}^N y_{ij}}{\sum_{j=1}^L \sum_{i=1}^N y_{ij}}$ represents frequency-based weight for label j . *F1* is the harmonic mean of *P* and *R* (higher is better), and weighted metrics are useful when label frequencies are imbalanced.

5) *Hamming Loss (HM)*: *HM* quantifies the fraction of labels that are incorrectly predicted across all instances and labels.

$$\text{HM} = \frac{1}{N \cdot L} \sum_{i=1}^N \sum_{j=1}^L \mathbb{I}(\tilde{y}_{ij} \neq y_{ij}) \quad (25)$$

Lower values are desirable, with zero indicating perfect label-wise predictions.

6) *Jaccard Similarity Score (J_s)*: J_s is a widely used metric in MLC that measures the overlap between the predicted and ground-truth positive label sets. It is defined as the size of the intersection divided by the size of the union of predicted and actual positive labels, computed per sample and averaged over the dataset.

$$J_s = \frac{1}{N} \sum_{i=1}^N \frac{|\tilde{y}_i \cap y_i|}{|\tilde{y}_i \cup y_i|} \quad (26)$$

where $\tilde{y}_i$ denotes the predicted positive label set and y_i denotes the ground-truth positive label set for instance i . Unlike *AvgAcc*, J_s considers only the positive labels. It ignores TN and penalizes both FP and FN, making it more robust in imbalanced settings.

7) *ROC-AUC Score*: The Area Under the Receiver Operating Characteristic Curve (ROC-AUC) evaluates the model's ability to distinguish positive and negative cases using probability scores $\hat{p}_{ij}$. Three variants are considered.

Micro-averaged aggregates predictions across all labels:

$$\text{ROC-AUC}_\mu = \text{AUC}(\text{Flatten}(y_{ij}), \text{Flatten}(\hat{p}_{ij})) \quad (27)$$

Macro-averaged computes AUC independently for each label and then averages across labels:

$$\text{ROC-AUC}_\mu = \frac{1}{L} \sum_{j=1}^L \text{AUC}(y_{\cdot j}, \hat{p}_{\cdot j}) \quad (28)$$

Weighted-averaged weights each label's AUC by its support:

$$\text{ROC-AUC}_\psi = \sum_{j=1}^L w_j \cdot \text{AUC}(y_{\cdot j}, \hat{p}_{\cdot j}) \quad (29)$$

Values range from 0.0 to 1.0, with higher scores indicating stronger discrimination between positive and negative cases.

D. Hyperparameter

Hyperparameter selection stabilizes the learning behavior of the LLM-DL models, especially when modeling source code with complex token, function, variable, and operator structures. Optuna-based tuning is performed for each LLM-DL combination to identify effective settings for the decoder and classifier components. The general search space and fixed settings used for tuning and final training are summarized in Table II³. The best-performing hyperparameter configurations for each architecture are listed in Table III. For each model combination, the best trial (*id*) is selected based on the validation weighted F1-score, denoted as $F1_\psi$ (*ts*). The tuned parameters include hidden dimension (*hd*), number of recurrent layers (*#L*), dropout rate (*dr*), learning rate (*lr*), batch size (*bs*), and bidirectional setting (*bd*). These parameters are optimized to improve MLC performance while reducing overfitting across different LLM-DL architectures.

VI. EXPERIMENTAL RESULTS

This section presents experimental results for the MLEC task on a real-world multi-label Python source code dataset. Sections VI-A through VI-F report, in turn, the 32 LLM-DL combinations, encoder baselines with a linear head, a seed-controlled comparison of decoder choice, pooling, label-modeling ablations, and computational cost.

³The *lr* range was adjusted only for RoBERTa_LR experiments.

TABLE II: Hyperparameter settings for the experiment

Hyperparameter	Search Space / Value
Tokenizer	Model-specific tokenizer
Output Layer	Linear (d_r, L), where $L = 11$
Hidden Dimension	{64, 128, 256}
Number of Layers	{1, 2} ; Dropout rate: [0.1, 0.3] (Uniform)
Maximum Sequence Length	256
Batch Size	{4, 8, 16}
Bidirectional	{True, False}
Optimizer	Adam ; Loss: BCEWithLogitsLoss
Activation Function	Sigmoid for probability conversion during evaluation
Learning Rate	[1e-5, 1e-1] (LogUniform)
Number of Epochs	20 for final training

TABLE III: The best hyperparameters of the models

Model Combination		id	ts	Best Hyperparameters					
				hd	#L	dr	lr	bs	bd
Code T5	GRU	8	0.7153	128	2	0.1260	$9.02e^{-5}$	8	F
	LSTM	2	0.7365	256	1	0.1827	$3.67e^{-5}$	8	T
	BiLSTM	2	0.7239	256	1	0.1134	$2.71e^{-5}$	4	T
	BiLSTM-A	7	0.6755	256	1	0.1138	$6.06e^{-4}$	4	T
Graph Code BERT	GRU	6	0.7272	128	1	0.1304	$2.51e^{-5}$	8	F
	LSTM	2	0.3802	128	2	0.1846	$2.43e^{-2}$	8	F
	BiLSTM	6	0.6181	128	2	0.2710	$1.16e^{-5}$	4	T
	BiLSTM-A	2	0.6367	256	2	0.1069	$5.16e^{-4}$	8	T
Code T5+	GRU	3	0.7249	128	2	0.1696	$2.29e^{-5}$	4	T
	LSTM	4	0.5026	128	1	0.2203	$2.66e^{-4}$	4	T
	BiLSTM	1	0.6601	128	2	0.2578	$2.98e^{-5}$	8	T
	BiLSTM-A	9	0.6382	256	2	0.1601	$4.89e^{-4}$	4	T
Uni Xcoder	GRU	0	0.6058	128	2	0.1475	$3.28e^{-4}$	4	F
	LSTM	6	0.5798	128	2	0.2334	$2.29e^{-4}$	4	F
	BiLSTM	6	0.6098	128	2	0.2747	$6.82e^{-4}$	4	T
	BiLSTM-A	3	0.5952	128	2	0.1475	$1.79e^{-3}$	16	T
Ro BERTa	GRU	4	0.6448	256	2	0.2323	$1.32e^{-5}$	4	T
	LSTM	0	0.3802	128	2	0.1941	$5.86e^{-2}$	16	F
	BiLSTM	7	0.5295	128	2	0.1844	$3.03e^{-5}$	8	T
	BiLSTM-A	3	0.2648	128	2	0.1306	$5.40e^{-3}$	4	T
Ro BERTa_LR	GRU	6	0.6516	64	1	0.1475	$1.52e^{-5}$	4	T
	LSTM	4	0.6572	256	1	0.2920	$1.48e^{-5}$	8	F
	BiLSTM	0	0.6124	128	1	0.2919	$1.08e^{-5}$	16	T
	BiLSTM-A	8	0.5731	128	2	0.1769	$1.40e^{-5}$	4	T
PL BART	GRU	5	0.6723	128	1	0.1776	$2.38e^{-5}$	4	T
	LSTM	2	0.6098	256	2	0.1057	$1.25e^{-5}$	4	F
	BiLSTM	8	0.6738	256	1	0.1411	$4.28e^{-5}$	16	T
	BiLSTM-A	5	0.6761	256	1	0.2825	$2.04e^{-5}$	16	T
Co Text	GRU	1	0.6878	256	1	0.2670	$1.28e^{-4}$	16	T
	LSTM	5	0.5790	256	1	0.1195	$2.55e^{-5}$	8	F
	BiLSTM	7	0.6498	128	2	0.2331	$9.06e^{-5}$	8	T
	BiLSTM-A	2	0.6156	64	2	0.1702	$2.58e^{-3}$	4	T

A. Performance of LLM-DL Combinations

Table IV, Table V, and Table VI give the combination results across the accuracy, label-level, and overlap/ranking metric groups defined in Section V. Table IV reports each combination on the accuracy metrics and *OE*. For **CodeT5**, all four configurations exceed 91% *AvgAcc*. BiLSTM yields the highest *AvgAcc* and *EMAcc*, 91.63% and 52.55%, with 8699 exact matches and an *OE* of 0.0710. **GraphCodeBERT** performs strongly with the GRU, BiLSTM, and BiLSTM-A variants: BiLSTM reaches the highest *AvgAcc* (91.61%) with an *OE* of 0.0774, and GRU the best *EMAcc* (53.19%) with 8806 exact matches. For **CodeT5+**, the GRU, BiLSTM, and BiLSTM-A variants all perform strongly, each above 90%

AvgAcc. GRU delivers the highest scores, with an *AvgAcc* of 91.84%, *EMAcc* of 53.78%, 8904 *#EM*, and an *OE* of 0.0708. **UniXcoder** records moderate results, with BiLSTM-A leading the group at 88.28% *AvgAcc*, 38.26% *EMAcc*, 6334 *#EM*, and an *OE* of 0.1415.

TABLE IV: Experimental results (*AvgAcc*, *EMAcc*, *#EM*, and *OE*) for the error understanding

Model Combination		<i>AvgAcc</i>	<i>EMAcc</i>	<i>#EM</i>	<i>OE</i>
CodeT5	GRU	0.9157	0.5241	8677	0.0748
	LSTM	0.9155	0.5223	8647	0.0742
	BiLSTM	0.9163	0.5255	8699	0.0710
	BiLSTM-A	0.9116	0.4964	8218	0.0788
Graph CodeBERT	GRU	0.9158	0.5319	8806	0.0788
	LSTM	0.7672	0.0230	381	0.4471
	BiLSTM	0.9161	0.5285	8749	0.0774
	BiLSTM-A	0.9108	0.4954	8201	0.0842
CodeT5+	GRU	0.9184	0.5378	8904	0.0708
	LSTM	0.7955	0.1105	1830	0.3860
	BiLSTM	0.9167	0.5279	8739	0.0722
	BiLSTM-A	0.9088	0.4907	8123	0.0893
UniXcoder	GRU	0.8786	0.3592	5946	0.1456
	LSTM	0.8816	0.3728	6171	0.1426
	BiLSTM	0.8800	0.3684	6099	0.1482
	BiLSTM-A	0.8828	0.3826	6334	0.1415
RoBERTa	GRU	0.9097	0.5029	8325	0.0926
	LSTM	0.7716	0.0780	1291	0.4471
	BiLSTM	0.7659	0.0998	1652	0.4471
	BiLSTM-A	0.7716	0.0780	1291	0.4471
RoBERTa_LR	GRU	0.9077	0.4936	8172	0.0948
	LSTM	0.9076	0.4971	8230	0.0955
	BiLSTM	0.9087	0.4964	8218	0.0946
	BiLSTM-A	0.9052	0.4830	7996	0.0971
PLBART	GRU	0.9001	0.4580	7583	0.1131
	LSTM	0.9023	0.4661	7717	0.1009
	BiLSTM	0.9009	0.4584	7589	0.1099
	BiLSTM-A	0.8988	0.4516	7476	0.1124
CoTexT	GRU	0.9058	0.4698	7777	0.0936
	LSTM	0.9014	0.4563	7554	0.0988
	BiLSTM	0.9034	0.4680	7747	0.0965
	BiLSTM-A	0.8774	0.3613	5981	0.1514

For **RoBERTa**, the GRU variant stands out at 90.97% *AvgAcc*, 50.29% *EMAcc*, 8325 exact matches, and an *OE* of 0.0926. **RoBERTa_LR**, incorporating *lr* optimization, is competitive across variants: BiLSTM reaches 90.87% *AvgAcc* with an *OE* of 0.0946, and LSTM the highest *EMAcc* (49.71%) with 8230 *#EM*. **PLBART** is stable across architectures, with LSTM best at 90.23% *AvgAcc*, 46.61% *EMAcc*, 7717 exact matches, and an *OE* of 0.1009. **CoTexT** performs comparably, its GRU variant reaching 90.58% *AvgAcc*, 46.98% *EMAcc*, 7777 *#EM*, and an *OE* of 0.0936. Across all combinations, CodeT5+_GRU is the strongest on these four metrics. Most encoders peak in a single variant, while GraphCodeBERT and RoBERTa_LR spread their best results across variants.

Table V reports *P*, *R*, and *F1* under the macro ($\bar{\mu}$) and weighted (ψ) settings. A consistent pattern holds across encoders: GRU delivers the best *F1* balance, while BiLSTM-A tends to maximize precision at the cost of recall. On **CodeT5+**, GRU leads both *F1* scores ($F1_{\bar{\mu}}$: 0.7602, $F1_{\psi}$: 0.8243), the highest in the table. GRU is also the best *F1* variant for **CodeT5** (0.8216), **GraphCodeBERT** (0.8198), **RoBERTa** (0.8018), and **CoTexT** (0.7970); **RoBERTa_LR** peaks with BiLSTM (0.7978) and **PLBART** with LSTM (0.7887). **UniX-**

TABLE V: Quantitative classification results of the *P*, *R*, and *F1* for error understanding

Model Combination		Precision		Recall		F1 Score	
		$P_{\bar{\mu}}$	P_{ψ}	$R_{\bar{\mu}}$	R_{ψ}	$F1_{\bar{\mu}}$	$F1_{\psi}$
CodeT5	GRU	0.7444	0.8334	0.7464	0.8116	0.7436	0.8216
	LSTM	0.7478	0.8336	0.7509	0.8069	0.7469	0.8196
	BiLSTM	0.7655	0.8348	0.7397	0.8099	0.7515	0.8215
	BiLSTM-A	0.7982	0.8533	0.6602	0.7552	0.7201	0.7999
Graph CodeBERT	GRU	0.7607	0.8388	0.7313	0.8021	0.7450	0.8198
	LSTM	0.0935	0.2032	0.1818	0.3929	0.1234	0.2676
	BiLSTM	0.7903	0.8509	0.7075	0.7836	0.7448	0.8154
	BiLSTM-A	0.8135	0.8534	0.6450	0.7507	0.7122	0.7973
CodeT5+	GRU	0.7897	0.8438	0.7337	0.8060	0.7602	0.8243
	LSTM	0.3651	0.5195	0.2272	0.4516	0.2252	0.4294
	BiLSTM	0.7736	0.8412	0.7332	0.7998	0.7522	0.8198
	BiLSTM-A	0.7948	0.8521	0.6513	0.7419	0.7144	0.7919
UniXcoder	GRU	0.7558	0.7994	0.4986	0.6471	0.5815	0.7087
	LSTM	0.7517	0.8175	0.4938	0.6391	0.5797	0.7114
	BiLSTM	0.7681	0.8235	0.4772	0.6244	0.5729	0.7003
	BiLSTM-A	0.7343	0.8026	0.5343	0.6665	0.6091	0.7247
RoBERTa	GRU	0.7785	0.8345	0.6859	0.7725	0.7272	0.8018
	LSTM	0.0503	0.1167	0.0909	0.2111	0.0647	0.1503
	BiLSTM	0.0929	0.2007	0.1818	0.3902	0.1228	0.2647
	BiLSTM-A	0.0503	0.1167	0.0909	0.2111	0.0647	0.1503
RoBERTa_LR	GRU	0.7777	0.8308	0.6803	0.7657	0.7233	0.7959
	LSTM	0.7582	0.8343	0.6942	0.7629	0.7239	0.7965
	BiLSTM	0.7658	0.8392	0.6843	0.7609	0.7221	0.7978
	BiLSTM-A	0.7782	0.8362	0.6608	0.7445	0.7127	0.7866
PLBART	GRU	0.7301	0.8129	0.6864	0.7552	0.7068	0.7825
	LSTM	0.7376	0.8106	0.6988	0.7687	0.7161	0.7887
	BiLSTM	0.7365	0.8104	0.6748	0.7597	0.7035	0.7835
	BiLSTM-A	0.7230	0.7966	0.6914	0.7707	0.7060	0.7829
CoTexT	GRU	0.7470	0.8174	0.7122	0.7783	0.7287	0.7970
	LSTM	0.7443	0.8107	0.6731	0.7651	0.7046	0.7867
	BiLSTM	0.7429	0.8120	0.7045	0.7755	0.7223	0.7931
	BiLSTM-A	0.7702	0.8144	0.4854	0.6210	0.5823	0.6973

coder trails the code-specialized encoders across these metrics. Overall, CodeT5+_GRU is again the strongest configuration on $F1_{\psi}$.

Figure 6 plots $F1_{\psi}$, which captures performance robustness across label frequency distributions, against *EMAcc*, which assesses strict multi-label correctness for all combinations. The CodeT5, GraphCodeBERT, and CodeT5+ variants with GRU or BiLSTM cluster at the top on both axes, indicating robust generalization with precise label sets. GraphCodeBERT with LSTM and the RoBERTa variants other than GRU degrade sharply (LSTM and BiLSTM-A are uniform in this case), reflecting an incompatibility between those encoder features and the recurrent dynamics. RoBERTa_LR is stable across both metrics, and CoTexT, PLBART, and UniXcoder hold moderate $F1_{\psi}$ with somewhat lower *EMAcc*. CodeT5+_GRU leads on both metrics⁴.

Table VI reports *HM*, J_s , and the three ROC-AUC variants. CodeT5+_GRU attains the best class-level values, with the lowest *HM* (0.0816), the highest J_s (0.7634), and an AUC_{ψ} of 0.9283. **CodeT5** is close behind and, in fact, holds the highest AUC_{ψ} in the table, its GRU variant reaching 0.9314, with BiLSTM giving the group's best *HM* (0.0837) and J_s (0.7610). **GraphCodeBERT** with GRU, BiLSTM, or BiLSTM-A also ranks strongly, with J_s above 0.75 and AUC_{ψ} above 0.92. **RoBERTa_LR** is the most stable of the remaining

⁴This combination also gives the highest *#EM*, 8904, in Table IV.

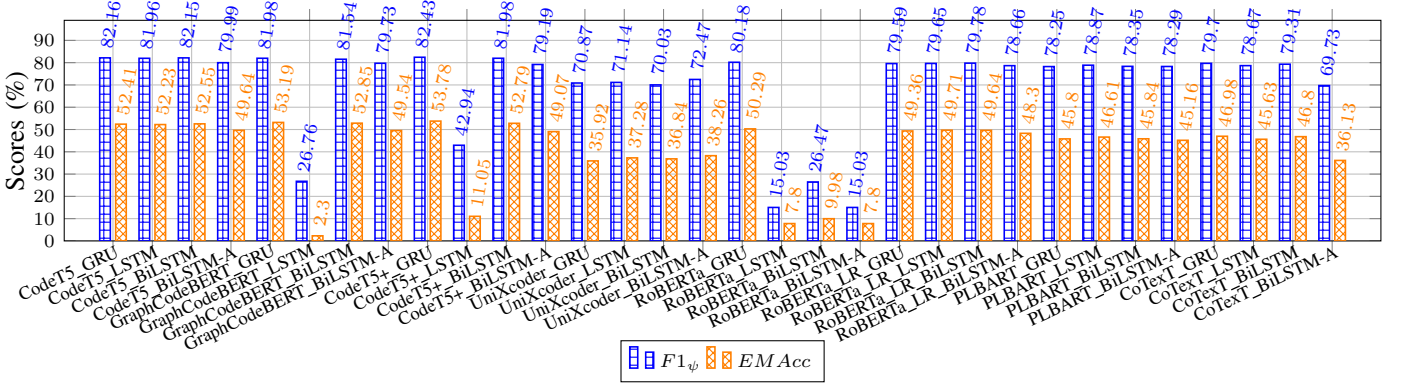


Fig. 6. $F1_\psi$ versus $EMAcc$ across all 32 combinations

TABLE VI: Evaluation of model combinations based on ROC-AUC, HM , and J_s

Model Combination		HM	J_s	ROC-AUC Score		
				$\bar{\mu}$	μ	ψ
CodeT5	GRU	0.0843	0.7580	0.9394	0.9558	0.9314
	LSTM	0.0845	0.7583	0.9286	0.9500	0.9258
	BiLSTM	0.0837	0.7610	0.9311	0.9514	0.9266
	BiLSTM-A	0.0884	0.7328	0.9386	0.9554	0.9283
GraphCodeBERT	GRU	0.0842	0.7588	0.9290	0.9506	0.9237
	LSTM	0.2328	0.3243	0.5000	0.7848	0.5000
	BiLSTM	0.0839	0.7556	0.9219	0.9481	0.9219
	BiLSTM-A	0.0892	0.7284	0.9351	0.9532	0.9248
CodeT5+	GRU	0.0816	0.7634	0.9326	0.9546	0.9283
	LSTM	0.2045	0.3802	0.7224	0.8329	0.6857
	BiLSTM	0.0833	0.7591	0.9318	0.9539	0.9276
	BiLSTM-A	0.0912	0.7231	0.9363	0.9534	0.9250
UniXcoder	GRU	0.1214	0.6201	0.9015	0.9303	0.8851
	LSTM	0.1184	0.6232	0.9043	0.9326	0.8901
	BiLSTM	0.1200	0.6107	0.9007	0.9305	0.8862
	BiLSTM-A	0.1172	0.6371	0.9057	0.9335	0.8905
RoBERTa	GRU	0.0903	0.7354	0.9286	0.9507	0.9213
	LSTM	0.2284	0.2489	0.5000	0.7718	0.5000
	BiLSTM	0.2341	0.3367	0.4840	0.7565	0.4947
	BiLSTM-A	0.2284	0.2489	0.5000	0.7869	0.5000
RoBERTa_LR	GRU	0.0923	0.7283	0.9235	0.9481	0.9174
	LSTM	0.0924	0.7287	0.9259	0.9481	0.9178
	BiLSTM	0.0913	0.7292	0.9252	0.9490	0.9186
	BiLSTM-A	0.0948	0.7161	0.9259	0.9478	0.9157
PLBART	GRU	0.0999	0.7063	0.9079	0.9386	0.9049
	LSTM	0.0977	0.7139	0.9159	0.9422	0.9080
	BiLSTM	0.0991	0.7090	0.9090	0.9384	0.9034
	BiLSTM-A	0.1012	0.7077	0.9082	0.9370	0.9011
CoText	GRU	0.0942	0.7217	0.9189	0.9439	0.9115
	LSTM	0.0986	0.7126	0.9293	0.9488	0.9173
	BiLSTM	0.0966	0.7198	0.9202	0.9455	0.9143
	BiLSTM-A	0.1226	0.6068	0.9012	0.9290	0.8820

encoders, its BiLSTM reaching AUC_ψ 0.9186. The collapsed runs, GraphCodeBERT_LSTM and the three RoBERTa variants other than GRU, reappear here with HM above 0.20 and AUC_ψ near 0.5. **PLBART**, **CoText**, and **UniXcoder** remain in the moderate range, with J_s between 0.61 and 0.72 and AUC_ψ above 0.88. Overall, the code-specialized encoders with GRU or BiLSTM give the strongest and most consistent label separability.

Figure 7 contrasts AUC_ψ , which measures label ranking under imbalance, with OE , which measures top-1 reliability. CodeT5 (GRU and BiLSTM-A) and CodeT5+_GRU give

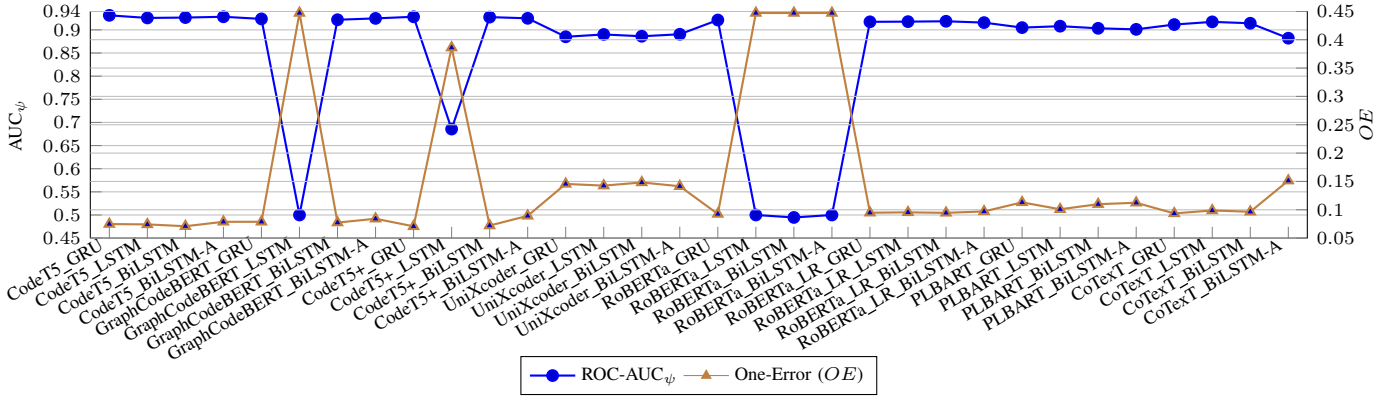
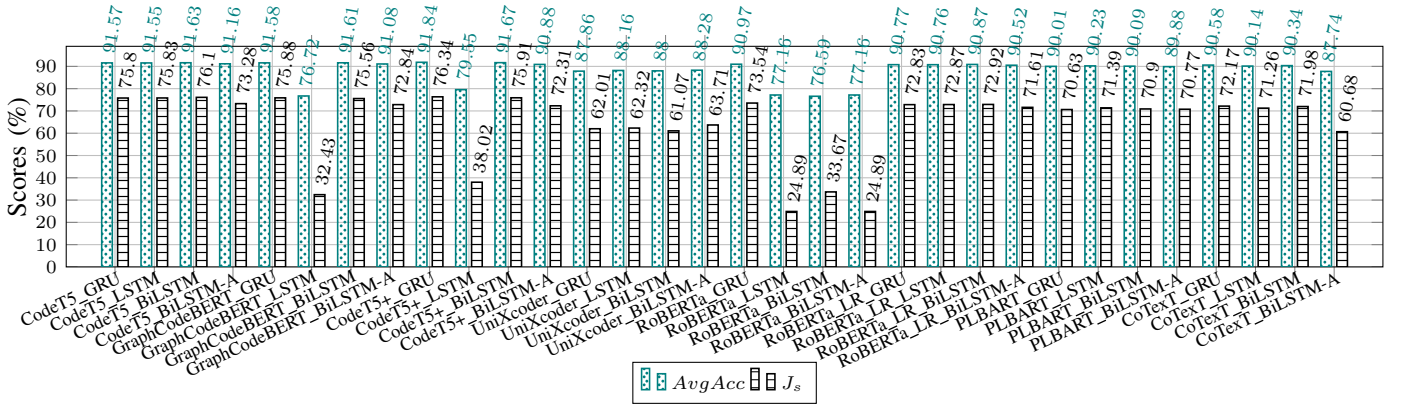
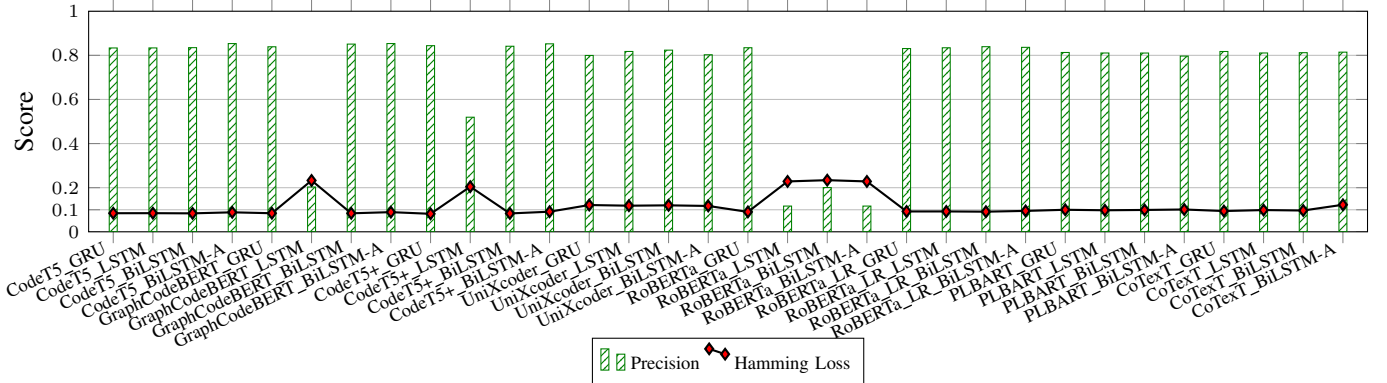
the best trade-off, pairing high AUC_ψ (up to 0.9314) with the lowest OE (0.0708). The RoBERTa variants other than GRU and GraphCodeBERT_LSTM sit at the opposite corner, with AUC_ψ near 0.5 and OE at 0.4471. These tied, chance-level values indicate optimization collapse rather than partial learning. The consistently low OE of the CodeT5, GraphCodeBERT, and CodeT5+ combinations confirms reliable top-1 predictions, in line with their AUC_ψ .

Figures 8, 9, and 10 extend the comparison to three further metric pairs. In Figure 8, $AvgAcc$ against J_s , the code-specialized encoders with GRU or BiLSTM again lead, and CodeT5+_GRU peaks on both ($AvgAcc$ 91.84%, J_s 76.34%). Figure 9 pairs P_ψ with HM : the code-specialized encoders, CodeT5, GraphCodeBERT, and CodeT5+, hold the highest precision, and CodeT5+_GRU records the lowest HM (0.0816), so strong precision coincides with low label-wise noise. Figure 10 then separates ranking from retrieval, AUC_μ against R_μ : discrimination stays uniformly high while recall varies more, with CodeT5_GRU highest on AUC_μ (95.58%) and CodeT5_LSTM broadest on recall (R_μ 75.09%). Across all three views, the code-specialized encoders paired with GRU or BiLSTM give the most balanced results, consistent with Table V and Table VI.

Table VII aligns the best combinations with the assessment themes. CodeT5+ and CodeT5 with GRU are effective across broad correctness, frequency-aware ranking, and retrieval precision. CodeT5_LSTM contributes the strongest recall (R_μ : 0.7509) and GraphCodeBERT_BiLSTM-A the highest precision (P_ψ : 0.8534). Across the themes, both encoder choice and recurrent structure shape label-level accuracy, structural consistency, and confidence in MLEC.

B. Encoder Contribution with a Linear Head

The LLM-DL combination results establish that the pipeline performs well, but they do not separate the share carried by the encoder from that carried by the recurrent decoder. To measure the encoder's own contribution, each of four representative encoders is paired with the linear classification head of (13), which removes the recurrent decoder while leaving tokenization, loss, optimizer, data split, and schedule unchanged. Table VIII reports the outcome across the metric

Fig. 7. AUC_ψ versus OE across all 32 combinationsFig. 8. $AvgAcc$ versus J_s across combinationsFig. 9. P_ψ versus HM across combinations

suite, with each configuration trained across multiple seeds and summarized as mean $\pm$ standard deviation. The CodeT5+ head selected a dropout of 0.22, a learning rate of $4.8e-5$, and a batch size of 16; the remaining heads were tuned by the same procedure.

With the head architecture fixed, the encoder alone moves $F1_\psi$ from 0.7846 to 0.8263, a span of 0.0417. The ordering tracks code specialization: the code-specialized encoders CodeT5 and CodeT5+ lead, GraphCodeBERT follows, and the general-purpose RoBERTa trails. CodeT5 and CodeT5+ form the top tier and are very close, with CodeT5 marginally ahead

on $F1_\psi$ (0.8263 against 0.8251) and CodeT5+ ahead on the majority of the remaining metrics. This spread, produced by the encoder while every other stage is held constant, indicates that the encoder is a decisive factor in MLEC performance.

C. Decoder Contribution under Matched Seeds

Table VIII isolates the encoder; the complementary question is how much the recurrent decoder adds on top of a strong encoder. To answer it on equal footing, the CodeT5+ linear head is compared directly with the CodeT5+ GRU hybrid, the strongest combination in Table IV, using the same five seeds

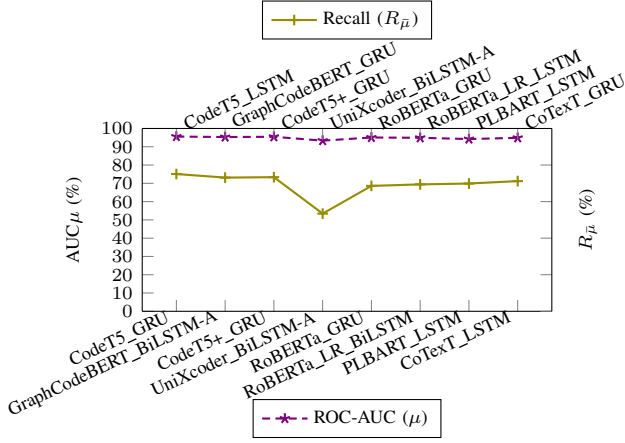


Fig. 10. ROC-AUC(μ) versus $R_{\bar{\mu}}$ for top-performing combinations

TABLE VII: Best-performing LLM-DL combinations across evaluation themes

Evaluation Theme	Metric(s)	Focus	Metric Value(s) with Best Combinations
Broad effectiveness	$F1_{\psi}$, $EM Acc$	Partial vs. complete correctness	$F1_{\psi}$: 0.8243 and $EM Acc$: 0.5378, CodeT5+_GRU
Frequency-Skewed ranking	ROC-AUC ψ , OE	Class dominance ranking	AUC ψ : 0.9314 and OE : 0.0708, CodeT5 and CodeT5+ with GRU
Structural prediction match	$Avg Acc$, J_s	Label set overlap	$Avg Acc$: 0.9184 and J_s : 0.7634, CodeT5+_GRU
Confidence vs. noise spread	P_{ψ} , HM	Trust vs. total error	P_{ψ} : 0.8534 and HM : 0.0816, GraphCodeBERT_BiLSTM-A and CodeT5+_GRU
Label retrieval and ranking	ROC-AUC(μ), $R_{\bar{\mu}}$	Retrieval precision	AUC(μ): 0.9558 and $R_{\bar{\mu}}$: 0.7509, CodeT5_GRU and CodeT5_LSTM

for both. Evaluated under this multi-seed protocol, the GRU hybrid reaches a mean $F1_{\psi}$ of 0.8212, close to but distinct from its single-run value of 0.8243 in Table V. Table IX reports the paired comparison.

On CodeT5+, the linear head performs on par with the GRU hybrid across the metric suite, holding a small and consistent edge on most measures and remaining level on $F1_{\bar{\mu}}$ and AUC ψ . On $F1_{\psi}$, the two are very close, 0.8251 against 0.8212, with the head slightly ahead on each of the five seeds ($p = 0.0013$). Read alongside Table VIII, the comparison points to where performance mainly comes from. With the head architecture fixed, the encoder alone shifts $F1_{\psi}$ by 0.0417 across the four encoders, and by 0.0145 among the code-specialized encoders, while changing the decoder on CodeT5+ shifts it by 0.0040. The encoder is the larger factor by a wide margin, and once it is strong, a simple head already captures most of what the hybrid achieves. This balance does shift with encoder strength. On CodeT5, the head reaches 0.8263 against 0.8216 for the reported hybrid, and on CodeT5+ 0.8251 against 0.8243, whereas on GraphCodeBERT and RoBERTa the head does not lead the corresponding hybrids in Table V. Since those hybrids are single runs and

TABLE VIII: Linear-head results by encoder (mean $\pm$ SD; five seeds for CodeT5+, three otherwise)

Metric	CodeT5	CodeT5+	Graph CodeBERT	RoBERTa
$F1_{\psi}$	0.8263 ± 0.0010	0.8251 ± 0.0012	0.8119 ± 0.0042	0.7846 ± 0.0048
$F1_{\bar{\mu}}$	0.7563 ± 0.0052	0.7600 ± 0.0040	0.7211 ± 0.0124	0.6559 ± 0.0081
$Avg Acc$	0.9184 ± 0.0016	0.9185 ± 0.0005	0.9089 ± 0.0028	0.8868 ± 0.0047
$EM Acc$	0.5345 ± 0.0030	0.5370 ± 0.0027	0.4962 ± 0.0130	0.4149 ± 0.0190
OE	0.0711 ± 0.0026	0.0708 ± 0.0013	0.0859 ± 0.0031	0.1158 ± 0.0058
HM	0.0816 ± 0.0016	0.0815 ± 0.0005	0.0911 ± 0.0028	0.1132 ± 0.0047
J_s	0.7656 ± 0.0014	0.7647 ± 0.0011	0.7443 ± 0.0069	0.6948 ± 0.0115
AUC ψ	0.9276 ± 0.0007	0.9287 ± 0.0014	0.9245 ± 0.0005	0.9197 ± 0.0014

TABLE IX: CodeT5+ linear head vs. GRU hybrid, five matched seeds (mean $\pm$ SD; * $p < 0.05$, paired t -test)

Metric	Linear head	GRU hybrid	p
$F1_{\psi}$	0.8251 ± 0.0012	0.8212 ± 0.0011	0.0013*
$F1_{\bar{\mu}}$	0.7600 ± 0.0040	0.7568 ± 0.0032	0.2726
$Avg Acc$	0.9185 ± 0.0005	0.9172 ± 0.0009	0.0130*
$EM Acc$	0.5370 ± 0.0027	0.5278 ± 0.0058	0.0146*
OE	0.0708 ± 0.0013	0.0724 ± 0.0015	0.0011*
HM	0.0815 ± 0.0005	0.0828 ± 0.0009	0.0130*
J_s	0.7647 ± 0.0011	0.7603 ± 0.0015	0.0003*
AUC ψ	0.9287 ± 0.0014	0.9276 ± 0.0004	0.1489

the head results are seeded means, this is best read as an observation rather than a tested difference, and it points to a natural reading: the recurrent decoder may matter more when the encoder’s representations are weaker, which would be worth a seed-matched study of its own.

D. Pooling Ablation

The linear head aggregates the encoder output by max pooling. To test whether the aggregation step itself matters, the maximum in (13) is replaced by mean pooling and by additive attention pooling, following (14), with all else unchanged. Each variant received its own hyperparameter search. The attention search converged on the same configuration as the max baseline, making that contrast a direct comparison of the pooling operator alone, while the mean search selected a lower lr . Table X reports the three variants. Max pooling gives the best result on every metric, and its lead over both alternatives holds on all five seeds. On $F1_{\psi}$, the margin is 0.0086 over mean pooling ($p = 0.0013$) and 0.0112 over attention pooling ($p = 0.0016$). The larger and learned attention pooling does not help here; the simplest operator is also the most effective. This is consistent with the combination results, where the BiLSTM and BiLSTM-A variants differ only in that BiLSTM-A replaces max pooling with additive attention pooling: across Table IV to Table VI, BiLSTM outperforms BiLSTM-A on six of the eight encoders on $F1_{\psi}$.

E. Modeling Label Dependencies

The heads above predict all labels from one shared representation, so they capture label dependence only implicitly. Section V showed that the labels in this dataset do co-occur, with the dependence concentrated among a few frequent pairs. The label-attention head of (15) models these dependencies

TABLE X: Pooling strategies on the CodeT5+ linear head (mean $\pm$ SD, five seeds)

Metric	Max	Mean	Attention
$F1_\psi$	0.8251 ± 0.0012	0.8165 ± 0.0017	0.8139 ± 0.0026
$F1_{\bar{\mu}}$	0.7600 ± 0.0040	0.7497 ± 0.0065	0.7453 ± 0.0060
$AvgAcc$	0.9185 ± 0.0005	0.9159 ± 0.0008	0.9143 ± 0.0007
$EMAcc$	0.5370 ± 0.0027	0.5254 ± 0.0037	0.5185 ± 0.0035
OE	0.0708 ± 0.0013	0.0775 ± 0.0013	0.0795 ± 0.0029
HM	0.0815 ± 0.0005	0.0841 ± 0.0008	0.0857 ± 0.0007
J_s	0.7647 ± 0.0011	0.7548 ± 0.0026	0.7507 ± 0.0028
AUC_ψ	0.9287 ± 0.0014	0.9233 ± 0.0010	0.9244 ± 0.0012

explicitly, letting the label embeddings interact through self-attention before classification. The selected configuration used $d_l = 64$ and $n_h = 4$. Table XI compares it with the plain CodeT5+ head under matched seeds. Explicit label interaction does not raise $F1_\psi$; the plain head stays marginally ahead on four of the five seeds. The remaining metrics divide along a clear line. The label-attention head lowers OE on every seed and slightly raises AUC_ψ , both of which rank labels without reference to the 0.5 threshold, while on the threshold-dependent metrics, $F1_{\bar{\mu}}$, $EMAcc$, and HM , it stays level with the plain head. This division follows the label distribution seen in Section V: the dependence available to model is concentrated in a few frequent pairs, and the rarest labels are close to independent, so the mechanism sharpens ranking among the frequent labels rather than completing full label sets. Modeling label correlations thus helps ranking quality on this dataset, even where it leaves thresholded classification unchanged.

TABLE XI: Label-attention vs. linear head on CodeT5+, five matched seeds (mean $\pm$ SD; * $p < 0.05$)

Metric	Linear head	Label-attention	p
$F1_\psi$	0.8251 ± 0.0012	0.8214 ± 0.0032	0.0664
$F1_{\bar{\mu}}$	0.7600 ± 0.0040	0.7353 ± 0.0125	0.0157*
$AvgAcc$	0.9185 ± 0.0005	0.9143 ± 0.0033	0.0637
$EMAcc$	0.5370 ± 0.0027	0.5158 ± 0.0164	0.0613
OE	0.0708 ± 0.0013	0.0680 ± 0.0011	0.0114*
HM	0.0815 ± 0.0005	0.0857 ± 0.0033	0.0637
J_s	0.7647 ± 0.0011	0.7567 ± 0.0066	0.0546
AUC_ψ	0.9287 ± 0.0014	0.9295 ± 0.0014	0.3977

F. Training and Inference Cost

Beyond accuracy, deployment in feedback systems depends on cost. Table XII reports training time and inference throughput for the leading configurations, measured on the hardware of Section V without concurrent jobs, so these figures are directly comparable to one another. On CodeT5+, the linear head trains 23.9% faster than the GRU hybrid, 394 minutes against 518, and processes 12.0% more samples per second at inference. Every head in the table, including the slowest, trains faster than the hybrid. The efficiency gain therefore comes at no accuracy cost on this encoder, since Table IX showed the head to be at least as accurate as the hybrid on every metric. The throughput figures reflect batched inference over the full test set and are a measure of bulk

TABLE XII: Training time and inference throughput (mean $\pm$ SD); throughput is batched over the 16,555-sample test set

Configuration	Seeds	Train (min)	Throughput (samples/s)
CodeT5+ head	5	394.49 ± 1.53	163.38 ± 1.81
CodeT5+ GRU hybrid	5	518.21 ± 2.66	145.92 ± 1.24
CodeT5 head	3	385.75 ± 3.61	164.08 ± 2.18
GraphCodeBERT head	3	401.74 ± 2.05	166.40 ± 1.43
RoBERTa head	3	481.53 ± 1.31	151.87 ± 0.28
Label-attention head	5	389.42 ± 3.76	162.02 ± 3.31

processing rather than single-request latency, which would be the relevant quantity for interactive per-submission feedback.

VII. DISCUSSION

This section interprets the combination results alongside the baseline and ablation experiments, then discusses hyperparameter effects, suitability, scalability, and limitations.

A. Performance Analysis

1) *Overview:* The MLEC task is assessed with a broad metric suite: P , R , and $F1$ ($\bar{\mu}$, ψ) for classification under label imbalance; $EMAcc$ and $AvgAcc$ for strict and relaxed correctness; HM and J_s for partial errors and label overlap; ROC-AUC (μ , $\bar{\mu}$, ψ) for ranking quality; and OE for top-label error. Together these cover discriminative capacity, robustness to imbalance, and generalization. Hyperparameters are tuned with Optuna over lr , hd , recurrent depth, dr , bs , and directionality. Among the 32 combinations, CodeT5+_GRU is the strongest, with the highest $F1_\psi$ (0.8243), the lowest HM (0.0816), and leading $AvgAcc$ (0.9184) and J_s (0.7634). CodeT5 and GraphCodeBERT combinations follow closely, RoBERTa improves markedly once its lr range is narrowed in the RoBERTa_LR variant, and PLBART, CoText, and UniXcoder remain in a moderate band. These results establish that the combinations perform strongly across the suite. The baseline and ablation experiments in Section VI were added to understand where that strength originates, and they point consistently to the encoder, as discussed next.

2) *Where Performance Originates:* Three complementary experiments locate the main source of MLEC performance. Pairing each encoder with a simple linear head (Table VIII) shows that the encoder alone moves $F1_\psi$ across a wide range, ordered by code specialization, with CodeT5 and CodeT5+ clearly ahead of the general-purpose RoBERTa. Comparing the CodeT5+ head with the CodeT5+_GRU hybrid under matched seeds (Table IX) shows that, on a strong encoder, the head performs on par with the hybrid, and that the encoder is by far the larger lever: holding the head fixed, the encoder shifts $F1_\psi$ by 0.0417 across encoders against 0.0040 for the decoder on CodeT5+. The two remaining ablations reinforce the same reading. Neither attention pooling (Table X) nor an explicit label-interaction head (Table XI) improves thresholded prediction over the simple max-pooled head. Across all three, once the encoder is strong, added decoder or head complexity brings little further gain.

This does not diminish the recurrent decoders, which perform strongly throughout Section VI and remain competitive on every encoder. Rather, it clarifies the design space: representational quality is the primary factor, and a lightweight head is a strong and economical default on a capable code encoder. The picture appears to shift for weaker encoders, where the head’s advantage over the hybrid is not observed and the decoder may contribute more, though confirming this would need a seed-matched study across encoders. For practitioners building MLEC systems, the most effective single choice is the encoder, after which a simple head recovers most of the achievable performance at lower cost.

3) Why Introduce RoBERTa_LR Alongside RoBERTa?:

A narrower lr range can be particularly effective for large transformer models such as RoBERTa, which are sensitive to training dynamics [70], [71]. The broad range ($1e-5$, $1e-1$) that worked well for CodeT5 and CodeT5+ was unstable for RoBERTa in combination with recurrent layers. In the RoBERTa_LSTM tuning trials, the per-trial validation $F1$ varied sharply, some trials reaching moderate scores while others collapsed toward zero, and the selected configuration gave a weak final result ($F1_\psi = 0.1503$, Table V). Restricting the range to ($1e-5$, $5e-5$) removed this instability: trials were consistent and the final $F1_\psi$ rose to 0.7965. This behavior, most pronounced for the LSTM variants, motivated the RoBERTa_LR variant, which narrows the range to stabilize training and improve generalization.

4) *Impact of Hyperparameter Tuning:* Tuning shaped the performance and convergence of the combinations. Using Optuna (Table III), several trends emerged: (i) lr : the standard range ($1e-5$ – $1e-1$) suited most models (for example, CodeT5+_GRU at $2.29e-5$), while RoBERTa needed the narrowed range ($1e-5$ – $5e-5$) for stability (Section VII-A3). (ii) $\#L$: one or two recurrent layers were tested, with no consistent advantage for the deeper setting across encoders. (iii) hd : moderate hidden sizes balanced capacity and overfitting, with GRU and LSTM favoring 128 and the bidirectional variants 256. (iv) dr : dropout between 0.1 and 0.25 worked best, with CodeT5+_GRU at 0.1696. (v) bs : selected batch sizes were 4, 8, or 16 depending on the combination, applied at final training. (vi) bd : the bidirectional setting varied across combinations and was not decisive for the GRU models, consistent with the encoder already supplying bidirectional context. Overall, performance depended on interactions among parameters, such as dropout with depth and lr with encoder, rather than any single setting.

5) *Why GRU is the Strongest Recurrent Decoder:* Among the four recurrent decoders, GRU is consistently the best across encoders in Table IV to Table VI. A plausible explanation is its streamlined gating, which offers a favorable balance of capacity and training efficiency: with fewer parameters than LSTM or the bidirectional variants, it generalizes well when paired with high-dimensional LLM embeddings and is less prone to overfitting on a task with limited signal per instance. Because the encoder already provides bidirectional context, the added directionality of the heavier variants yields little here, which is consistent with the bidirectional setting not being decisive for the top GRU models. CodeT5+_GRU il-

lustrates this, pairing CodeT5+’s Python-oriented embeddings with a compact two-layer GRU. This advantage should be read in proportion, however: Section VI-C shows the decoder’s overall contribution is small next to the encoder’s, so GRU is best among decoders whose collective effect is secondary to encoder choice.

6) *Training and Optimization Time:* Total experimental time varied across the 32 combinations (Figure 11). GRU and BiLSTM-A variants generally trained in moderate time, while LSTM and BiLSTM variants tended to take longer, reflecting differences in model complexity. These durations were measured with concurrent GPU jobs and are therefore indicative rather than controlled; the directly comparable training and inference measurements for the leading configurations are reported separately in Section VI-F. The trend nonetheless underlines the value of balancing computational cost against architectural depth in practical MLEC systems.

B. Suitability, Scope, and Scalability

The LLM-DL approach studied here suits MLEC tasks in code-level settings, where label sparsity and semantic ambiguity are common. Its modular design, a pretrained encoder followed by a lightweight head or recurrent decoder, could adapt to other programming languages such as Java or C++ with minor tokenizer or architectural changes. The label-agnostic metrics suggest a fit for educational platforms and tutoring systems, where the models could help novice programmers identify and understand errors across large repositories in academia, industry, or OJ systems, and for SE tasks such as code review, fault localization, and refactoring. On scalability, the cost results in Section VI-F offer practical guidance: a simple head on a strong code encoder reaches competitive accuracy while training faster than the recurrent hybrid, which makes it an efficient default when throughput or latency is a concern. The hybrid remains a sound choice where its sequence modeling is valuable, for instance with weaker encoders, and distillation or pruning could reduce cost further in either case. The framework could also extend to related tasks such as algorithm or function classification under multi-label or hierarchical schemes.

C. Limitations

While the approach shows strong empirical performance, several limitations should be noted. (i) Seeded evaluation with paired testing covers the baseline and ablation experiments but not all 32 combinations, which follow the original single-run protocol; the cross-encoder head-versus-hybrid comparison is therefore reported as an observation. (ii) Evaluation uses a single Python error dataset, which may limit generalization to other languages or domains. (iii) The fixed Optuna budget may not reach globally optimal configurations, and performance may vary under more imbalanced or noisy label distributions. (iv) The all-combination timing in Figure 11 was measured with concurrent GPU jobs, so those durations are indicative rather than controlled. (v) Architectures beyond the tested set might behave differently. Broader validation across datasets, architectures, and tasks would strengthen these findings.

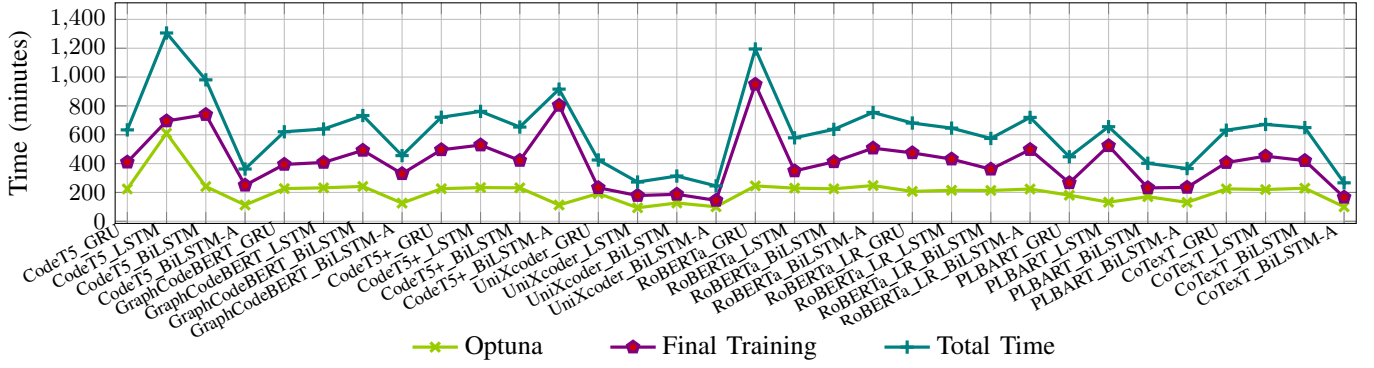


Fig. 11. Experimental duration across the 32 combinations

VIII. CONCLUSION

This study systematically evaluated 32 LLM-DL combinations for MLEC, pairing eight pretrained encoders with four recurrent decoders under a comprehensive metric suite and Optuna tuning. Among the combinations, CodeT5+_GRU performed best, reaching an $F1_\psi$ of 0.8243, $AvgAcc$ of 91.84%, $EMAcc$ of 53.78%, and HM of 0.0816. To identify the source of this performance, the study added seed-controlled baselines and component ablations with paired testing. These experiments point to the encoder as the primary factor: across encoders sharing a single linear head, $F1_\psi$ spans 0.7846 to 0.8263, ordered by code specialization, a range far wider than the effect of the decoder. On a strong code encoder, the simple head matches the recurrent hybrid while training faster, and neither attention pooling nor explicit label-interaction modeling improves thresholded prediction over it. The recurrent decoders remain competitive throughout and appear more relevant when the encoder is weaker. Together, these results clarify where MLEC performance originates and offer practical guidance for building efficient error-classification systems: encoder choice is the decisive lever, and a lightweight head is a strong, economical default on a capable encoder. Future work includes seed-matched comparisons across encoders, additional decoder and head designs, extension to further programming languages, and adaptation to related tasks such as defect detection, algorithm classification, and educational feedback systems.

REFERENCES

- [1] Y.-T. Lin, M. K.-C. Yeh, and S.-R. Tan, "Teaching programming by revealing thinking process: Watching experts' live coding videos with reflection annotations," *IEEE Transactions on Education*, vol. 65, no. 4, pp. 617–627, 2022.
- [2] I. Aldalur and X. Sagarna, "Improving programming learning in engineering students through discovery learning," *IEEE Revista Iberoamericana de Tecnologías del Aprendizaje*, 2023.
- [3] Y. Watanobe, M. M. Rahman, T. Matsumoto, U. K. Rage, and P. Ravikumar, "Online judge system: Requirements, architecture, and experiences," *International Journal of Software Engineering and Knowledge Engineering*, vol. 32, no. 06, pp. 917–946, 2022.
- [4] I. Mekterović, L. Brkić, B. Milašinović, and M. Baranović, "Building a comprehensive automated programming assessment system," *IEEE access*, vol. 8, pp. 81154–81172, 2020.
- [5] M. F. I. Amin, A. Shirafuji, M. M. Rahman, and Y. Watanobe, "Multi-label code error classification using codet5 and ml-knn," *IEEE Access*, 2024.
- [6] A. Shirafuji, T. Matsumoto, M. F. I. Amin, and Y. Watanobe, "Rule-based error classification for analyzing differences in frequent errors," in *2023 IEEE International Conference on Teaching, Assessment and Learning for Engineering (TALE)*, pp. 1–7, IEEE, 2023.
- [7] M. F. I. Amin, Y. Watanobe, M. M. Rahman, and A. Shirafuji, "Source code error understanding using bert for multi-label classification," *IEEE Access*, 2025.
- [8] H. Wan, H. Luo, M. Li, and X. Luo, "Automated program repair for introductory programming assignments," *IEEE Transactions on Learning Technologies*, 2024.
- [9] Z. Bian, Q. Chang, J. Wang, W. Pedrycz, and N. R. Pal, "Takagi-sugeno-kang fuzzy systems for high-dimensional multi-label classification," *IEEE Transactions on Fuzzy Systems*, 2024.
- [10] Z. Liu, C. Tang, S. E. Abhadiomhen, X.-J. Shen, and Y. Li, "Robust label and feature space co-learning for multi-label classification," *IEEE Transactions on Knowledge and Data Engineering*, vol. 35, no. 11, pp. 11846–11859, 2023.
- [11] C. Peng, H. Wang, J. Wang, L. Shou, K. Chen, G. Chen, and C. Yao, "Learning label-adaptive representation for large-scale multi-label text classification," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 2024.
- [12] Q. Chen, J. Du, A. Allot, and Z. Lu, "Litmc-bert: transformer-based multi-label classification of biomedical literature with an application on covid-19 literature curation," *IEEE/ACM transactions on computational biology and bioinformatics*, vol. 19, no. 5, pp. 2584–2595, 2022.
- [13] Z.-B. Yu and M.-L. Zhang, "Multi-label classification with label-specific feature generation: A wrapped approach," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 9, pp. 5199–5210, 2021.
- [14] S. F. Yilmaz, E. B. Kaynak, A. Koç, H. Dibeklioğlu, and S. S. Kozat, "Multi-label sentiment analysis on 100 languages with dynamic weighting for label imbalance," *IEEE Transactions on Neural Networks and Learning Systems*, 2021.
- [15] W. Zhou, Y. Hou, D. Chen, H. Hu, and T. Su, "Attention-augmented memory network for image multi-label classification," *ACM Transactions on Multimedia Computing, Communications and Applications*, vol. 19, no. 3, pp. 1–24, 2023.
- [16] G. Tsoumakas and I. Katakis, "Multi-label classification: An overview," *Data Warehousing and Mining: Concepts, Methodologies, Tools, and Applications*, pp. 64–74, 2008.
- [17] S. Minaee, N. Kalchbrenner, E. Cambria, N. Nikzad, M. Chenaghlu, and J. Gao, "Deep learning-based text classification: a comprehensive review," *ACM computing surveys (CSUR)*, vol. 54, no. 3, pp. 1–40, 2021.
- [18] M.-L. Zhang, Y.-K. Li, X.-Y. Liu, and X. Geng, "Binary relevance for multi-label learning: an overview," *Frontiers of Computer Science*, vol. 12, pp. 191–202, 2018.
- [19] E. Hancer, B. Xue, and M. Zhang, "A many-objective diversity-guided differential evolution algorithm for multi-label feature selection in high-dimensional datasets," *IEEE Transactions on Emerging Topics in Computational Intelligence*, 2025.
- [20] J. Huang, W. Qian, C.-M. Vong, W. Ding, W. Shu, and Q. Huang, "Multi-label feature selection via label enhancement and analytic hierarchy process," *IEEE Transactions on Emerging Topics in Computational Intelligence*, vol. 7, no. 5, pp. 1377–1393, 2023.
- [21] J. M. Moyano, E. L. Gibaja, K. J. Cios, and S. Ventura, "Review of ensembles of multi-label classifiers: Models, experimental study and prospects," *Information Fusion*, vol. 44, pp. 33–45, 2018.

- [22] W. Ni, K. Zhang, X. Miao, X. Zhao, Y. Wu, Y. Wang, and J. Yin, "Zeroed: Hybrid zero-shot error detection through large language model reasoning," *arXiv preprint arXiv:2504.05345*, 2025.
- [23] A. Shirafuji, Y. Watanobe, T. Ito, M. Morishita, Y. Nakamura, Y. Oda, and J. Suzuki, "Exploring the robustness of large language models for solving programming problems," *arXiv preprint arXiv:2306.14583*, 2023.
- [24] S. Gao, C. Gao, Y. He, J. Zeng, L. Nie, X. Xia, and M. Lyu, "Code structure-guided transformer for source code summarization," *ACM Transactions on Software Engineering and Methodology*, vol. 32, no. 1, pp. 1–32, 2023.
- [25] J. Liu, F. Zhang, X. Zhang, Z. Yu, L. Wang, Y. Zhang, and B. Guo, "hmcodetrans: Human-machine interactive code translation," *IEEE Transactions on Software Engineering*, vol. 50, no. 5, pp. 1163–1181, 2024.
- [26] M. Izadi, J. Katzy, T. Van Dam, M. Otten, R. M. Popescu, and A. Van Deursen, "Language models for code completion: A practical evaluation," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, pp. 1–13, 2024.
- [27] S. Fatima, H. Hemmati, and L. Briand, "Flakyfix: Using large language models for predicting flaky test fix categories and test code repair," *IEEE Transactions on Software Engineering*, 2024.
- [28] H. Joshi, J. C. Sanchez, S. Gulwani, V. Le, G. Verbruggen, and I. Radiček, "Repair is nearly generation: Multilingual program repair with llms," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, pp. 5131–5140, 2023.
- [29] A. Shirafuji, M. M. Rahman, M. F. I. Amin, and Y. Watanobe, "Program repair with minimal edits using codet5," in *2023 12th International Conference on Awareness Science and Technology (iCAST)*, pp. 178–184, IEEE, 2023.
- [30] M. Ma, G. Chochlakis, N. M. Pandiyan, J. Thomason, and S. Narayanan, "Large language models do multi-label classification differently," *arXiv preprint arXiv:2505.17510*, 2025.
- [31] J. Chen, R. Zhang, J. Xu, C. Hu, and Y. Mao, "A neural expectation-maximization framework for noisy multi-label text classification," *IEEE Transactions on Knowledge and Data Engineering*, vol. 35, no. 11, pp. 10992–11003, 2022.
- [32] Z. Wang, Z. Zhou, Y. H. Da Song, S. Chen, L. Ma, and T. Zhang, "Towards understanding the characteristics of code generation errors made by large language models," in *Proceedings of the IEEE/ACM 47th International Conference on Software Engineering (ICSE'25)*, 2025.
- [33] M. M. Rahman, A. Shirafuji, and Y. Watanobe, "Big coding data: Analysis, insights, and applications," *IEEE Access*, 2024.
- [34] Y. Watanobe, M. M. Rahman, M. F. I. Amin, and R. Kabir, "Identifying algorithm in program code based on structural features using cnn classification model," *Applied Intelligence*, vol. 53, no. 10, pp. 12210–12236, 2023.
- [35] M. M. Rahman and Y. Watanobe, "Multilingual program code classification using n -layered bi-lstm model with optimized hyperparameters," *IEEE Transactions on Emerging Topics in Computational Intelligence*, vol. 8, no. 2, pp. 1452–1468, 2023.
- [36] L. Chen, Y. Pei, M. Pan, T. Zhang, Q. Wang, and C. A. Furia, "Program repair with repeated learning," *IEEE Transactions on Software Engineering*, vol. 49, no. 2, pp. 831–848, 2022.
- [37] D. M. Muepu, Y. Watanobe, and M. F. I. Amin, "A comprehensive content-based recommendation system for programming problems through multi-faceted code analysis," *IEEE Access*, 2025.
- [38] M. S. Mia, Y. Watanobe, M. M. Rahman, M. F. I. Amin, and D. M. Muepu, "Attention-driven clustering of programming problem difficulty using statistical and machine learning techniques," *IEEE Access*, 2025.
- [39] Q. Wu, M. Tan, H. Song, J. Chen, and M. K. Ng, "MI-forest: A multi-label tree ensemble method for multi-label classification," *IEEE transactions on knowledge and data engineering*, vol. 28, no. 10, pp. 2665–2680, 2016.
- [40] J.-Y. Hang and M.-L. Zhang, "Dual perspective of label-specific feature learning for multi-label classification," *ACM Transactions on Knowledge Discovery from Data*, vol. 19, no. 1, pp. 1–30, 2024.
- [41] F. Li, Y. Zuo, H. Lin, and J. Wu, "Boostxml: gradient boosting for extreme multilabel text classification with tail labels," *IEEE Transactions on Neural Networks and Learning Systems*, 2023.
- [42] C. Si, Y. Jia, R. Wang, M.-L. Zhang, Y. Feng, and C. Qu, "Multi-label classification with high-rank and high-order label correlations," *IEEE Transactions on Knowledge and Data Engineering*, vol. 36, no. 8, pp. 4076–4088, 2023.
- [43] J. Wu, W. Gan, H. Lu, and P. S. Yu, "Graph contrastive learning on multi-label classification for recommendations," *ACM Transactions on Intelligent Systems and Technology*, vol. 16, no. 4, pp. 1–19, 2025.
- [44] H. Ye, R. Sunderraman, and S. Ji, "Matchxml: an efficient text-label matching framework for extreme multi-label text classification," *IEEE Transactions on Knowledge and Data Engineering*, 2024.
- [45] D. Mahapatra, A. J. Yepes, B. Bozorgtabar, S. Roy, Z. Ge, and M. Reyes, "Corrections to "multi-label generalized zero shot chest x-ray classification by combining image-text information with feature disentanglement"," *IEEE transactions on medical imaging*, vol. 44, no. 4, pp. 1984–1985, 2025.
- [46] M. Shao, A. Basit, R. Karri, and M. Shafique, "Survey of different large language model architectures: Trends, benchmarks, and challenges," *IEEE Access*, 2024.
- [47] Y. Wang, W. Wang, S. Joty, and S. C. Hoi, "Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation," *arXiv preprint arXiv:2109.00859*, 2021.
- [48] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, pp. 4171–4186, 2019.
- [49] D. Guo, S. Ren, S. Lu, Z. Feng, D. Tang, S. Liu, L. Zhou, N. Duan, A. Svyatkovskiy, S. Fu, et al., "Graphcodebert: Pre-training code representations with data flow," *arXiv preprint arXiv:2009.08366*, 2020.
- [50] Y. Wang, H. Le, A. D. Gotmare, N. D. Bui, J. Li, and S. C. Hoi, "Codet5+: Open code large language models for code understanding and generation," *arXiv preprint arXiv:2305.07922*, 2023.
- [51] A. Radford, K. Narasimhan, T. Salimans, I. Sutskever, et al., "Improving language understanding by generative pre-training," 2018.
- [52] D. Guo, S. Lu, N. Duan, Y. Wang, M. Zhou, and J. Yin, "Unixcoder: Unified cross-modal pre-training for code representation," *arXiv preprint arXiv:2203.03850*, 2022.
- [53] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "Roberta: A robustly optimized bert pretraining approach," *arXiv preprint arXiv:1907.11692*, 2019.
- [54] W. U. Ahmad, S. Chakraborty, B. Ray, and K.-W. Chang, "Unified pre-training for program understanding and generation," *arXiv preprint arXiv:2103.06333*, 2021.
- [55] B. Roziere, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan, Y. Adi, J. Liu, R. Sauvestre, T. Remez, et al., "Code llama: Open foundation models for code," *arXiv preprint arXiv:2308.12950*, 2023.
- [56] L. Phan, H. Tran, D. Le, H. Nguyen, J. Anibal, A. Peltekian, and Y. Ye, "Cotext: Multi-task learning with code-text transformer," *arXiv preprint arXiv:2105.08645*, 2021.
- [57] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [58] D. Bahdanau, K. Cho, and Y. Bengio, "Neural machine translation by jointly learning to align and translate," *arXiv preprint arXiv:1409.0473*, 2014.
- [59] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.
- [60] H. McNichols, M. Zhang, and A. Lan, "Algebra error classification with large language models," in *International Conference on Artificial Intelligence in Education*, pp. 365–376, Springer, 2023.
- [61] Y. Sun, Z. Yin, X. Huang, X. Qiu, and H. Zhao, "Error classification of large language models on math word problems: A dynamically adaptive framework," *arXiv preprint arXiv:2501.15581*, 2025.
- [62] S. Abbas, S. Ojo, M. Krichen, M. A. Alamro, A. Mihoub, and L. Vilcekova, "A novel deep learning approach for myocardial infarction detection and multi-label classification," *IEEE Access*, 2024.
- [63] Q. Zhang, C. Fang, Y. Xie, Y. Zhang, Y. Yang, W. Sun, S. Yu, and Z. Chen, "A survey on large language models for software engineering," *arXiv preprint arXiv:2312.15223*, 2023.
- [64] T. Chen, T. Wu, D. Pan, J. Xie, J. Zhi, X. Wang, L. Quan, and Q. Lyu, "Transnam: identifying twelve types of rna modifications by an interpretable multi-label deep learning model based on transformer," *IEEE/ACM Transactions on Computational Biology and Bioinformatics*, vol. 20, no. 6, pp. 3623–3634, 2023.
- [65] M. M. Rahman, A. I. Shiplu, Y. Watanobe, and M. A. Alam, "Roberta-bilstm: A context-aware hybrid model for sentiment analysis," *IEEE Transactions on Emerging Topics in Computational Intelligence*, 2025.
- [66] M. M. Rahman, Y. Watanobe, and K. Nakamura, "Source code assessment and classification based on estimated error probability using attentive lstm language model and its application in programming education," *Applied Sciences*, vol. 10, no. 8, p. 2973, 2020.
- [67] Y. Watanobe, "Aizu online judge," 2018.
- [68] Aizu Online Judge, "Developers Site (API)." Online, 2025. Accessed: June. 5, 2025.

- [69] G. Lyu, Z. Yang, X. Deng, and S. Feng, “L-vsm: Label-driven view-specific fusion for multiview multilabel classification,” *IEEE Transactions on Neural Networks and Learning Systems*, 2024.
- [70] X. Liu and C. Wang, “An empirical study on hyperparameter optimization for fine-tuning pre-trained language models,” *arXiv preprint arXiv:2106.09204*, 2021.
- [71] M. Mosbach, M. Andriushchenko, and D. Klakow, “On the stability of fine-tuning bert: Misconceptions, explanations, and strong baselines,” *arXiv preprint arXiv:2006.04884*, 2020.